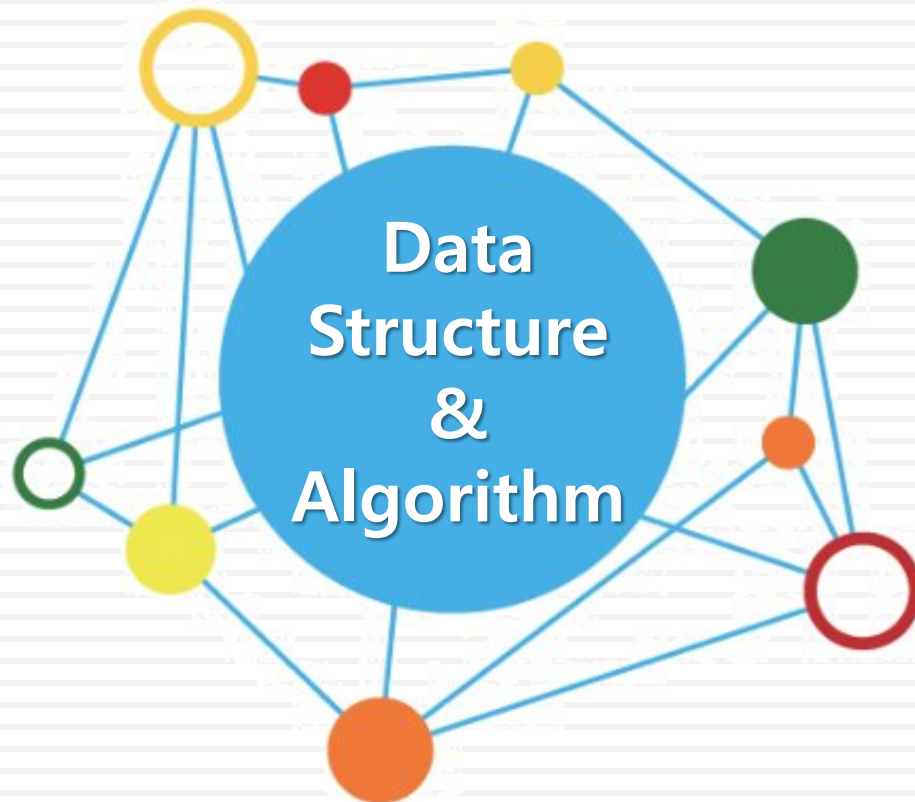


데이터 구조론



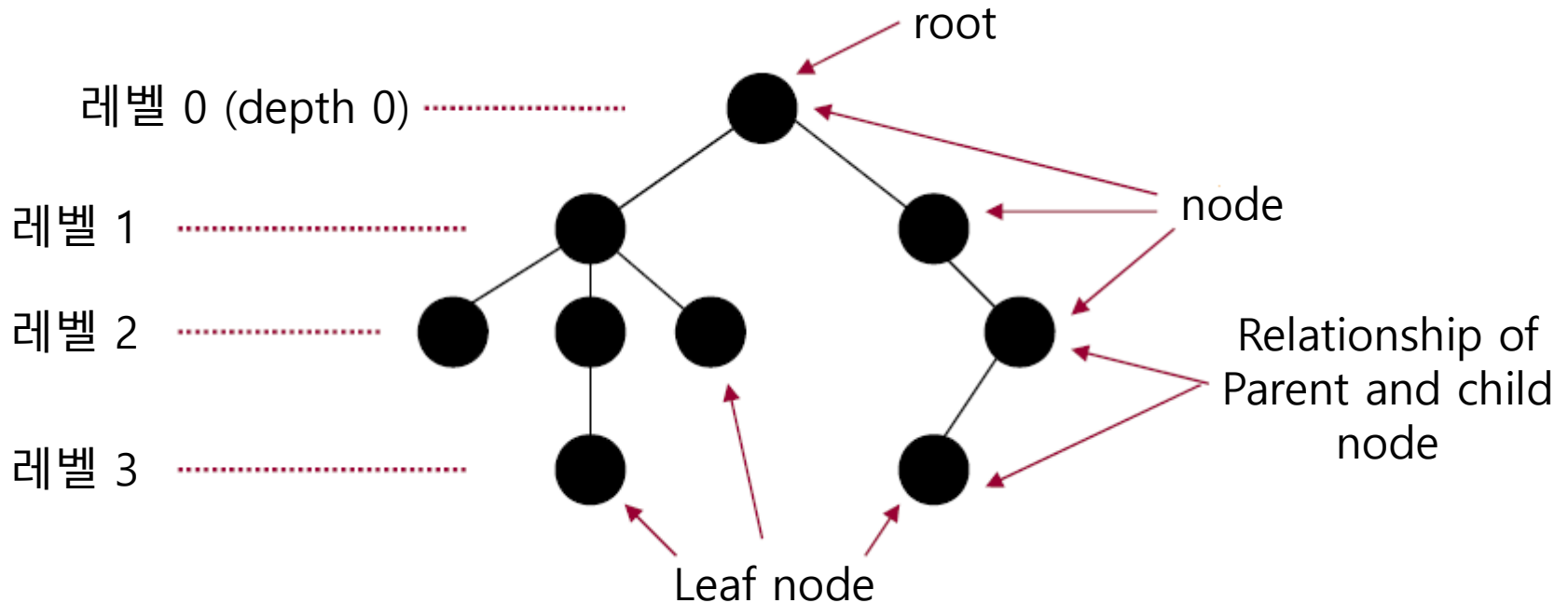
유길현
트리(Tree)

❖ 트리(Tree)란

- 원소들 간에 1:n 관계를 갖는 비선형 자료 구조
- 원소들 간에 계층관계를 갖는 계층형 자료구조
- 상위 원소에서 하위 원소로 내려가면서 확장되는 나무 모양의 구조
- 트리는 하나 이상의 노드(node)를 구성된 유한 집합으로 다음 두 가지 조건을 만족한다.
 - ① 특별히 지정된 루트(root)라고 부르는 특별한 노드가 있다.
 - ② 다른 노드들은 원소가 중복되지 않는 n 개의 서브 트리(subtree) $T_1, T_2, T_3, \dots, T_n$ 으로 나누어지며 T_i 각각은 **루트의 서브 트리**라고 한다.
- 이진 트리의 경우 산술적 표현이나 자료 구조 등을 매우 간단하게 표현할 수 있는 장점이 있음
- 컴퓨터 기술의 발전과 더불어 수많은 응용 분야에 적용 가능

❖ 트리의 구조와 구성요소

- 트리의 구조 : 모양이 뒤집어 놓은 나무와 같다.



❖ 트리의 구조와 구성요소

■ 트리의 구성요소

용 어	설 명
노드(Node)	트리의 원소
루트 노드(Root Node)	트리의 시작 노드, 부모가 없는 노드(레벨 0)
단말 노드(Leaf Node)	자식이 없는 노드, 말단 노드
간선(Edge)	노드를 연결하는 선
형제 노드(Sibling Node)	같은 부모를 갖는 노드
조상 노드(Ancessor Node)	간선을 따라 루트 노드까지 경로에 있는 모든 노드들
서브 트리(Subtree)	부모 노드와 연결된 간선을 끊었을 때 생성되는 트리
자손 노드(Descendant Node)	서브 트리에 있는 하위 레벨의 노드들
차수(Degree)	노드의 차수 : 노드에 연결된 자식 노드의 수 트리의 차수 : 트리에 있는 노드의 차수 중에서 가장 큰 값
높이(Hight)	노드의 높이 : 루트에서 노드에 이르는 간선의 수, 노드의 레벨 트리의 높이 : 트리에 있는 노드의 높이 중에서 가장 큰 값
레벨(Level)	트리의 특정 높이를 가지는 노드의 집합

❖ 트리의 특징

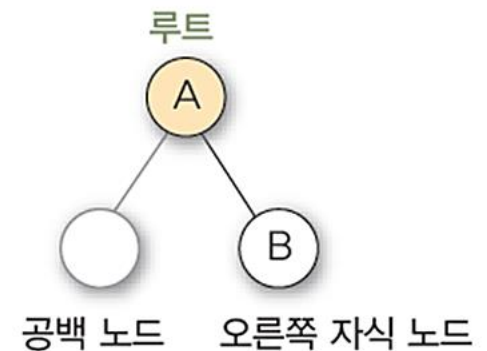
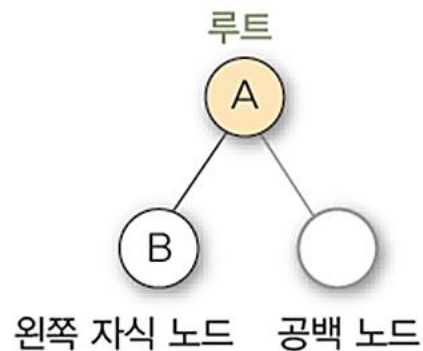
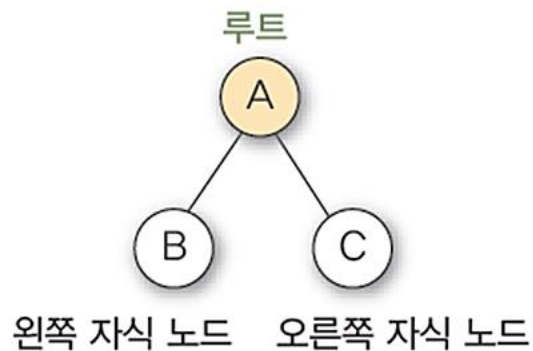
- 트리는 **계층 모델** 이다.
- 노드가 N개인 트리는 항상 N-1개의 간선(edge)을 가진다.
- 루트에서 어떤 노드로 가는 경로는 유일하다.
- 한 개의 루트 노드만이 존재하며 모든 자식 노드는 한 개의 부모 노드만을 가진다.
- 순회는 전위 순회, 중위 순회 아니면 후위 순회로 이루어진다. 이 3가지 모두 DFS/BFS 안에 있다.
- 트리는 이진 트리, 이진 탐색 트리, 균형 트리(AVL 트리, red-black 트리), 이진 힙(최대힙, 최소힙) 등이 있다.

DFS(Depth-First Search) : 깊이 우선 탐색

BFS(Breadth-First Search) : 너비 우선 탐색

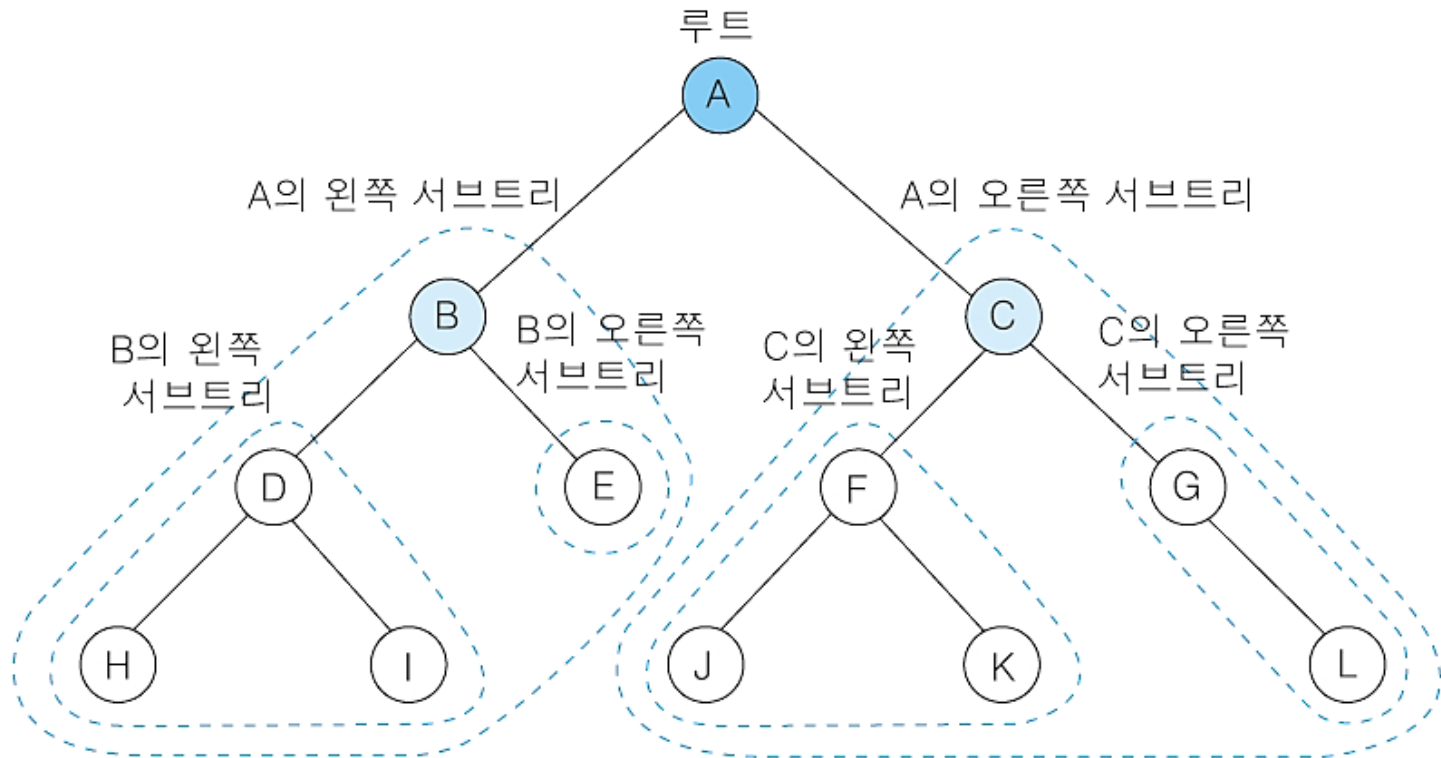
❖ 이진트리(Binary Tree) 란?

- 트리의 모든 노드의 차수를 2 이하로 제한하여 전체 트리의 차수가 2 이하가 되도록 정의
- 이진 트리의 모든 노드는 왼쪽 자식 노드와 오른쪽 자식 노드만 가짐
 - 부모 노드와 자식 노드 수와의 관계 📖 1:2
 - 공백 노드도 자식 노드로 취급
 - $0 \leq \text{노드의 차수} \leq 2$
- 이진 트리의 기본 구조



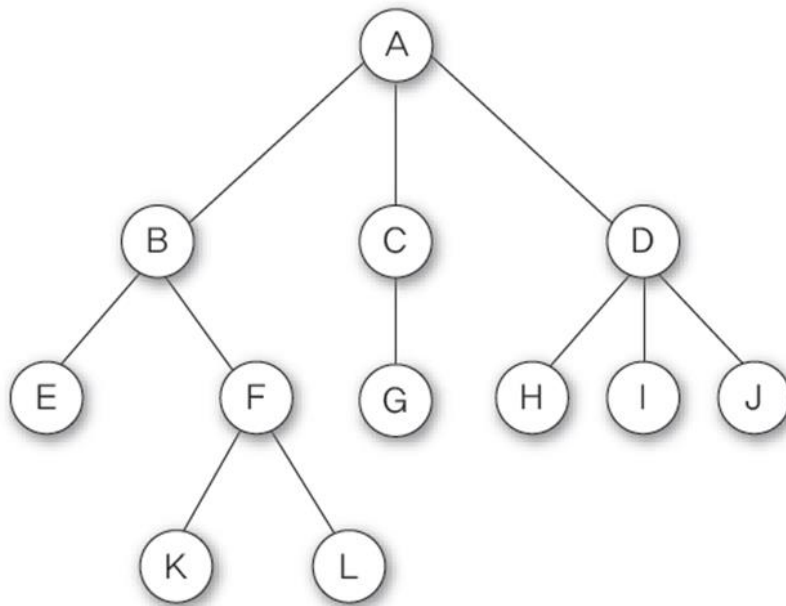
❖ 이진 트리 순환적 구성

- 노드의 왼쪽 자식 노드를 루트로 하는 왼쪽 서브트리도 이진 트리
- 노드의 오른쪽 자식 노드를 루트로 하는 오른쪽 서브 트리도 이진 트리

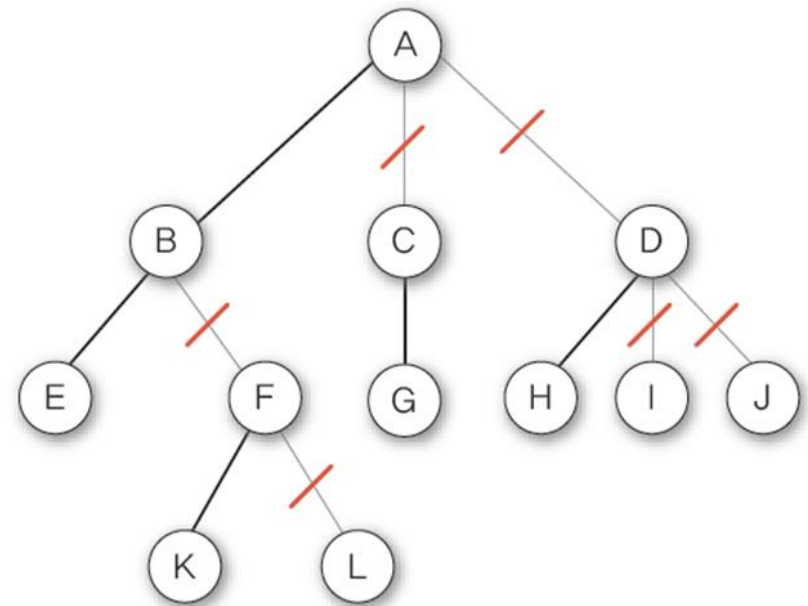


❖ 일반 트리를 이진 트리로 변환

- 이진 트리가 아닌 일반 트리는 자료 구조로 구현하는데 어려움이 있고 이진 트리 보다는 비효율적 이다.

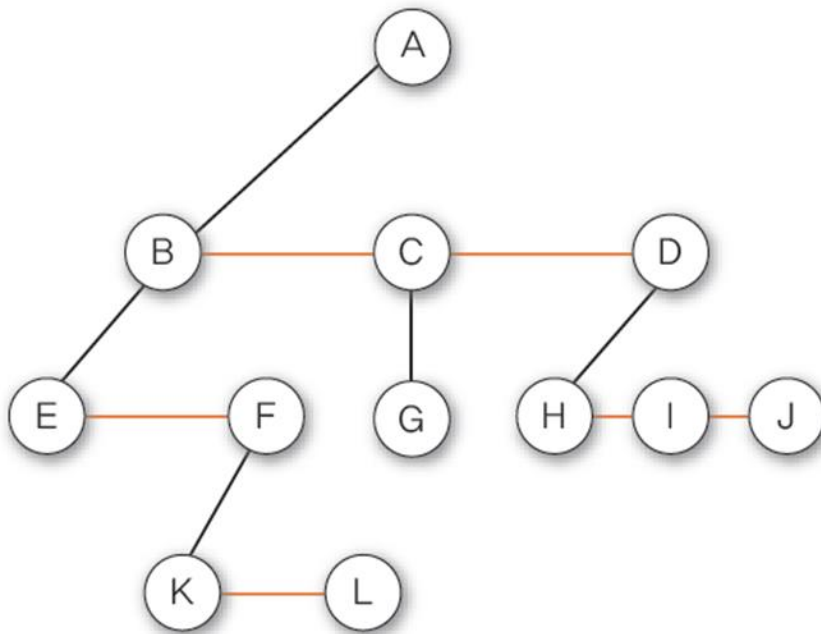


① 일반트리 A

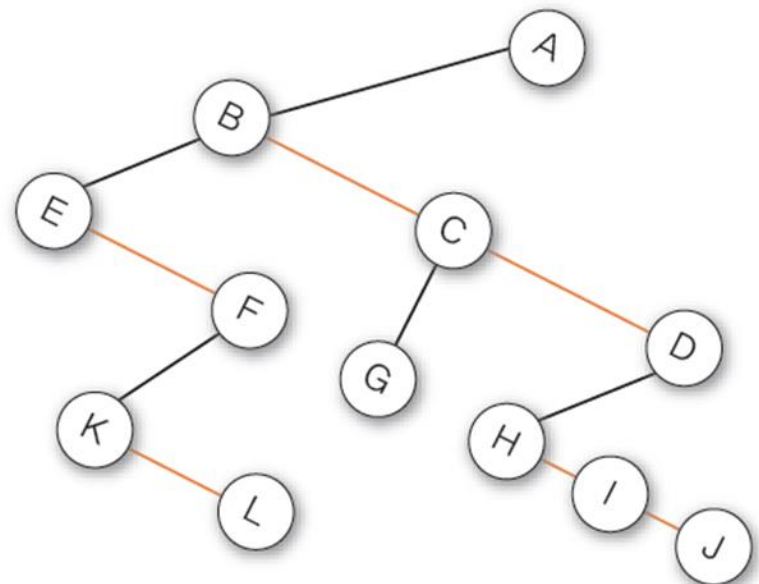


② 첫 번째 자식 노드 간선만 남기고 나머지 간선 제거

❖ 일반 트리를 이진 트리로 변환



③ 형제 노드를 간선으로 연결



④ 시계 방향으로 45° 회전

❖ 추상 자료형

ADT	이진 트리의 추상 자료형
ADT BinaryTree 데이터 : 공백이거나 루트 노드, 왼쪽 서브 트리, 오른쪽 서브 트리로 구성된 노드들의 유한 집합 연산 :	
bt, bt1, bt2 ∈ BinaryTree; item ∈ Element	
// 공백 이진 트리를 생성하는 연산 createBT() ::= create an empty binary tree;	
// 이진 트리가 공백인지 확인하는 연산 isEmpty ::= if(bi isEmpty) then return true else return false;	
// 두 개의 서브 트리를 연결하여 하나로 만드는 연산 makeBT(item, bt1, bt2) ::= return { item을 루트로 하고, bt1을 왼쪽 서브 루트, bt2를 오른쪽 서브 루트로 하는 이진 트리 }	

❖ 추상 자료형

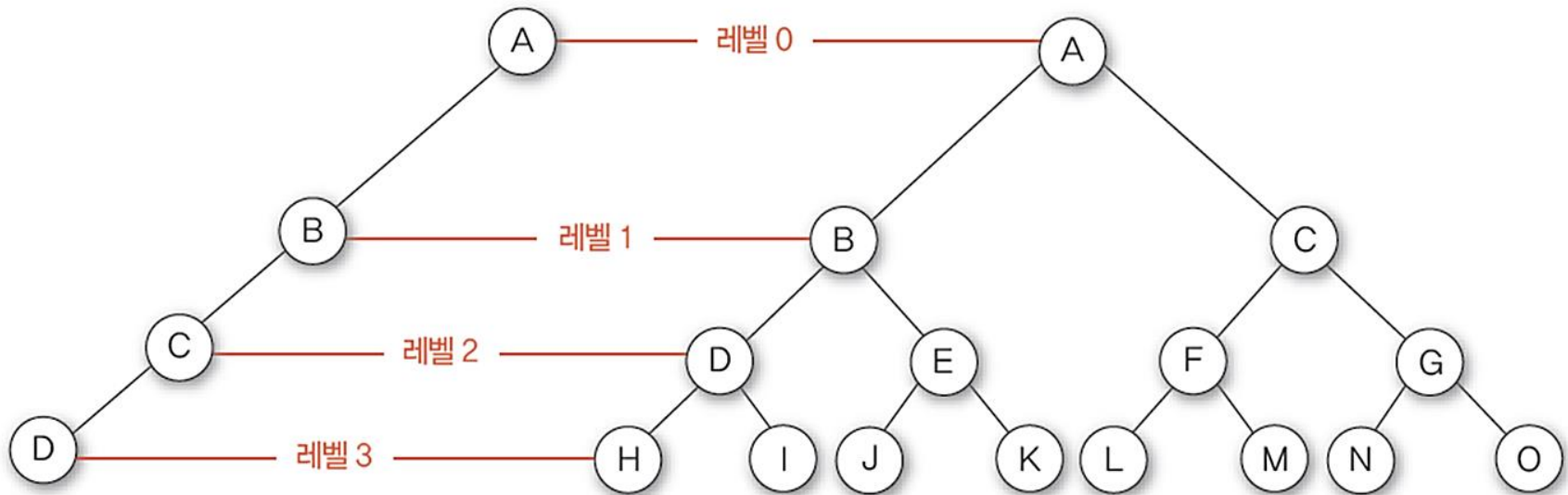
ADT	이진 트리의 추상 자료형
	<pre>// 이진 트리의 왼쪽 서브 트리를 구하는 연산 leftSubtree(bt) ::= if(isEmpty(bt) then return NULL else return subtree of bt; // 이진 트리의 오른쪽 서브 트리를 구하는 연산 rightSubtree(bt) ::= if (isEmpty(bt) then return NULL else return right subtree of bt; // 이진 트리에서 루트 노드의 데이터(item)를 구하는 연산 data(bt) ::= if (isEmpty(bt)) then return NULL else return the item in the root node of bt; End BinaryTree</pre>

❖ 이진 트리의 특성

- 정의 1) 노드가 n 개인 이진 트리는 항상 간선이 $(n-1)$ 개임
 - 루트를 제외한 $(n-1)$ 개의 노드가 부모 노드와 연결되는 한 개의 간선을 가짐
- 정의 2) 높이가 h 인 이진 트리가 가질 수 있는 노드 개수는 최소 $(h+1)$ 개이며, 최대 $(2^{h+1}-1)$ 개
 - 이진 트리의 높이가 h 가 되려면 한 레벨에 최소한 한 개의 노드가 있어야 하므로 높이가 h 인 이진 트리의 최소 노드의 개수는 $(h+1)$ 개
 - 하나의 노드는 최대 2개의 자식 노드를 가질 수 있으므로 레벨 i 에서의 노드의 최대 개수는 2^i 개 이므로 높이가 h 인 이진 트리 전체의 노드 개수는 $\sum_{i=0}^h 2^i = 2^{h+1}-1$ 개

❖ 이진 트리의 특성

- 높이가 3이면서 최소 노드를 갖는 이진 트리와 최대 노드를 갖는 이진 트리



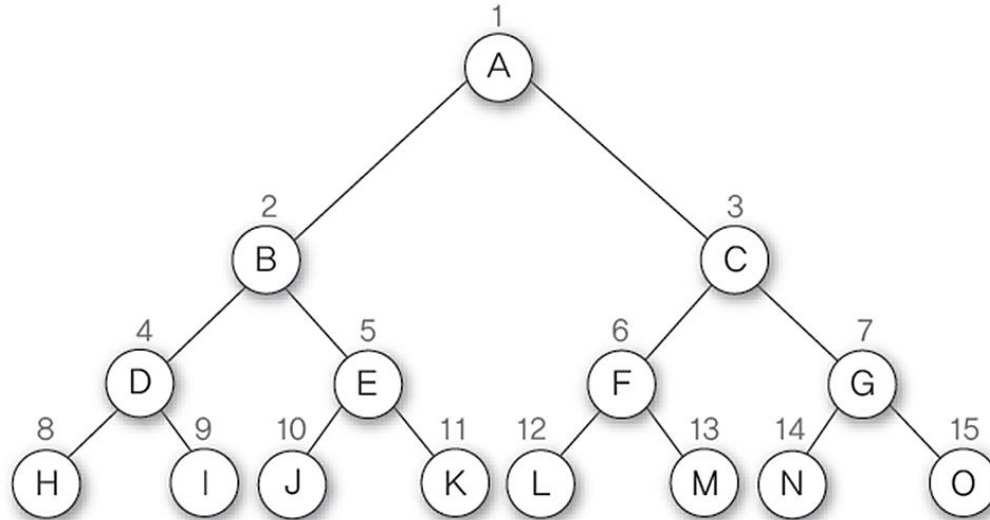
최소 노드를 갖는 이진 트리

최대 노드를 갖는 이진 트리

❖ 이진 트리의 종류

■ 포화 이진 트리(Full Binary Tree)

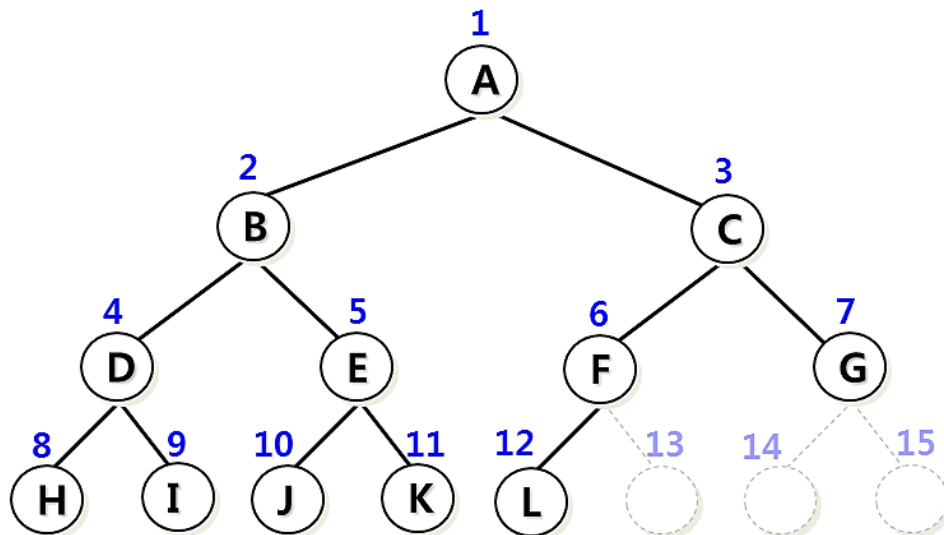
- 모든 레벨에 노드가 포화 상태로 차 있는 이진 트리
- 높이가 h 일 때, 최대의 노드 개수인 $(2^{h+1}-1)$ 의 노드를 가진 이진 트리
- 루트를 1번으로 하여 $2^{h+1}-1$ 까지 정해진 위치에 대한 노드 번호를 가짐



높이가 3인 포화 이진 트리

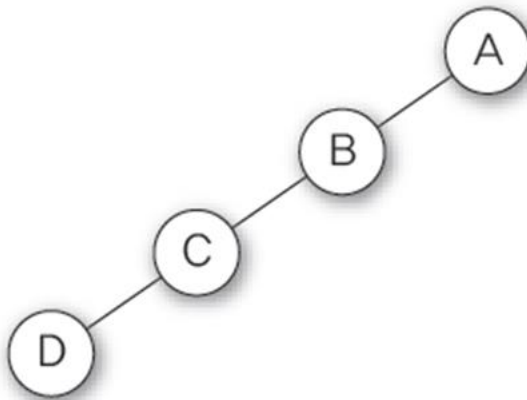
❖ 이진 트리의 종류

- 완전 이진 트리(Complete Binary Tree)
 - 높이가 h 이고 노드 수가 n 개일 때 (단, $n < 2^{h+1}-1$), 노드 위치가 포화 이진 트리에서의 노드 1번부터 n 번까지의 위치와 완전히 일치하는 이진 트리
 - 완전 이진 트리에서는 $(n+1)$ 번부터 $(2^{h+1}-1)$ 번까지 노드는 모두 공백 노드
 - 높이가 3인 완전 이진 트리

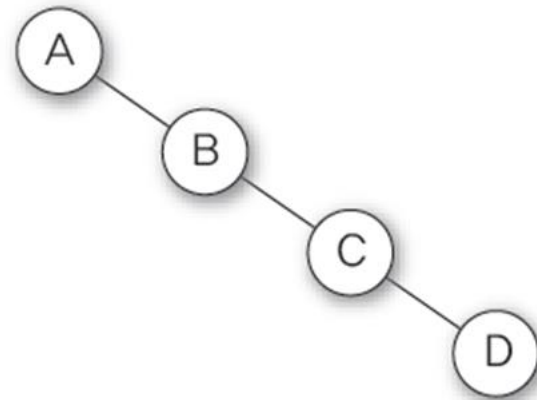


❖ 편향 이진 트리(Skewed Binary Tree)

- 높이가 h 일 때 $h+1$ 개의 노드를 가지면서 모든 노드가 왼쪽이나 오른쪽 중 한 방향으로만 서브 트리를 가지고 있는 트리
- 높이가 3인 편향 이진 트리



왼쪽 편향 이진 트리



오른쪽 편향 이진 트리

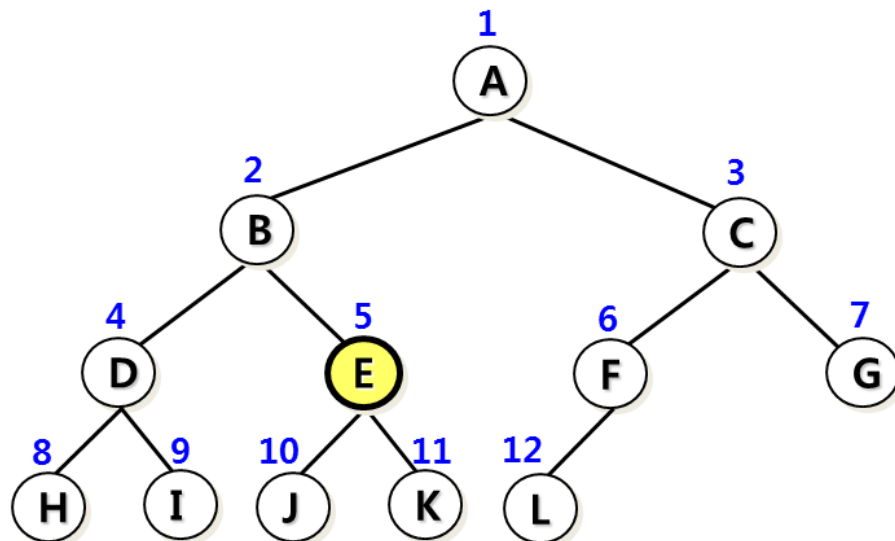
❖ 1차원 배열의 순차 자료구조 사용

- 높이가 h 인 포화 이진 트리의 노드번호를 배열의 인덱스로 사용
- 인덱스 0번 : 실제로 사용하지 않고 비워둠.
- 인덱스 1번 : 루트 저장
- 이진 트리를 표현한 1차원 배열에서의 인덱스 관계

노 드	인덱스	성립 조건
노드 i 의 부모 노드	$i \div 2$	$i > 1$
노드 i 의 왼쪽 자식 노드	$2 \times i$	$(2 \times i) \leq n$
노드 i 의 오른쪽 자식 노드	$(2 \times i) + 1$	$(2 \times i + 1) \leq n$
루트 노드	1	$n > 0$

순차 자료 구조를 이용한 이진 트리의 구현

❖ 완전 이진 트리의 1차원 배열 표현

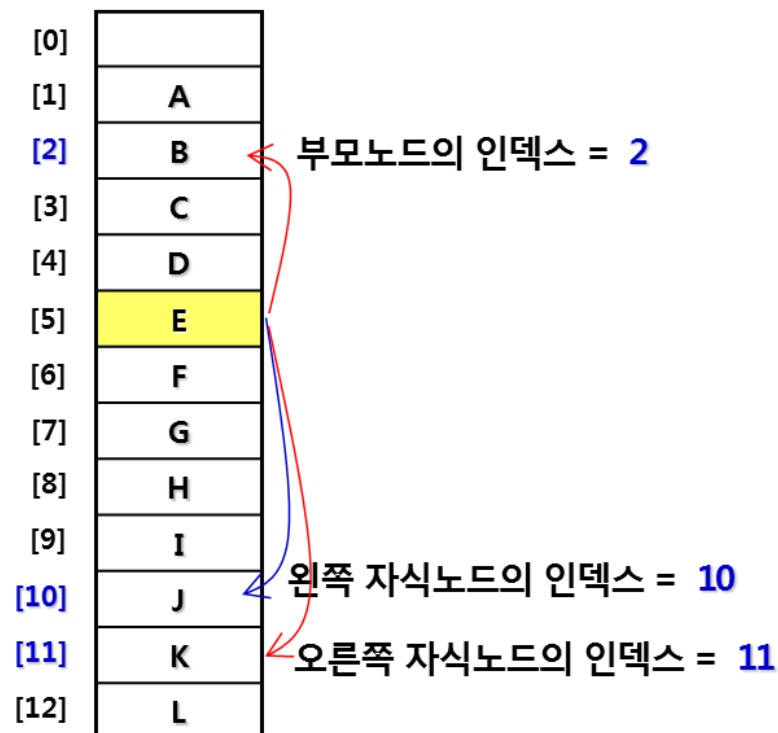


노드 i 의 부모 노드 : $i \div 2$

노드 i 의 왼쪽 자식 노드 : $2 \times i$

노드 i 의 오른쪽 자식 노드 : $(2 \times i) + 1$

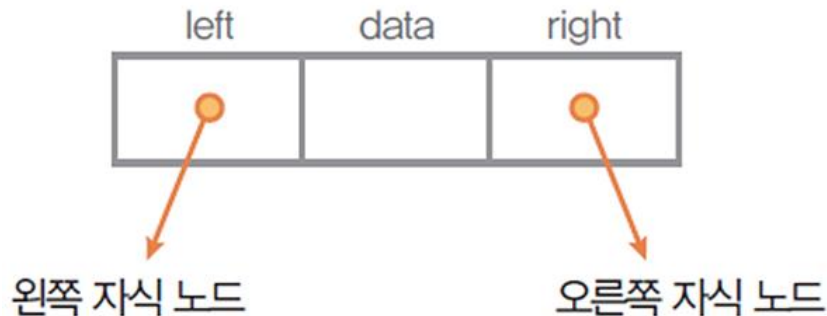
루트 노드 : 1



연결 자료 구조를 이용한 이진 트리의 구현

❖ 포인터를 사용하여 이진 트리 구현

- 데이터를 저장하는 데이터 필드, 왼쪽 자식 노드를 연결하는 왼쪽 링크 필드, 오른쪽 자식 노드를 연결하는 오른쪽 링크 필드로 구성. 자식 노드가 없으면 링크 필드에 NULL을 저장하여 NULL 포인터로 설정



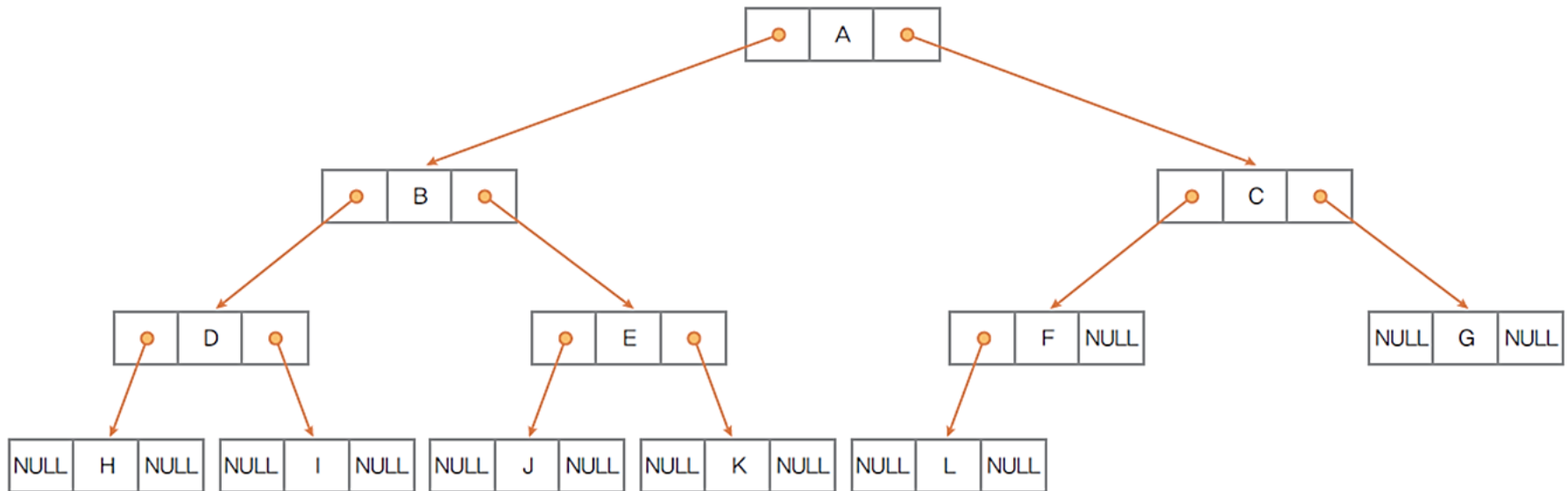
이진 트리 노드의 구조

```
typedef struct treeNode {  
    char data;  
    struct treeNode *left;  
    struct treeNode *right;  
} treeNode;
```

이진 트리 노드의 C구조체 정의

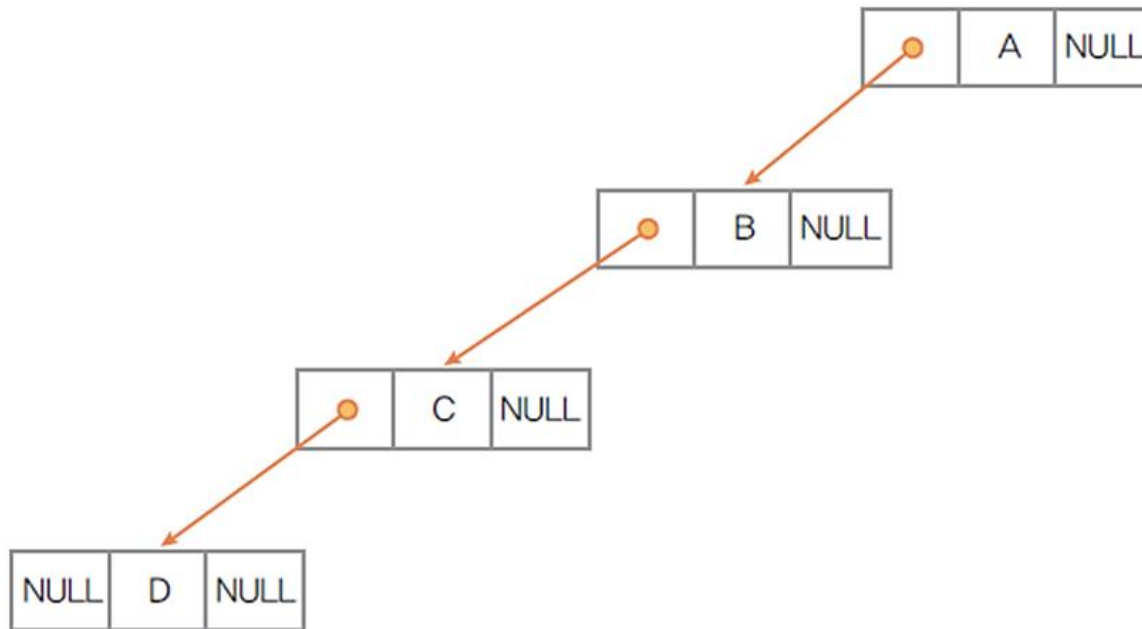
연결 자료 구조를 이용한 이진 트리의 구현

❖ 완전 이진 트리를 연결 자료구조 형태로 표현



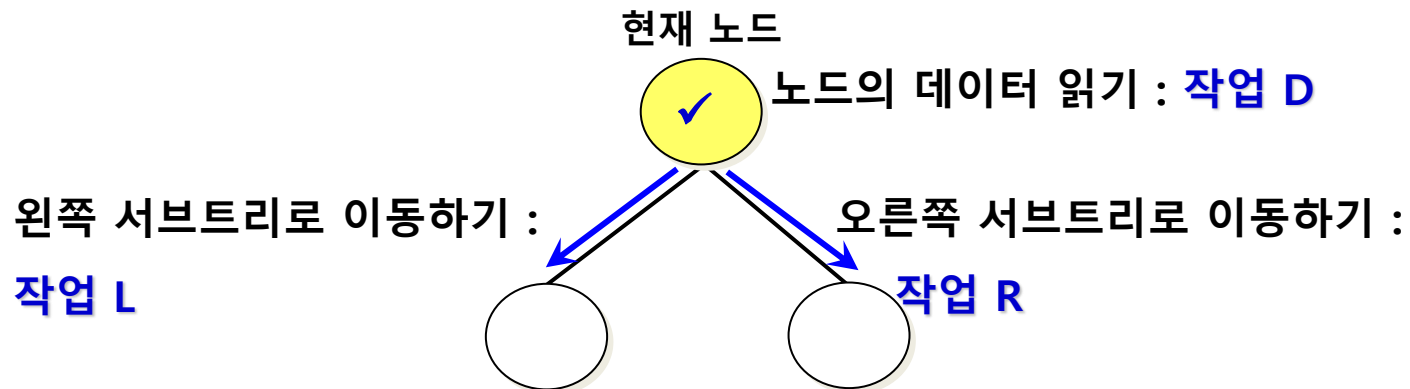
연결 자료 구조를 이용한 이진 트리의 구현

❖ 편향 이진 트리를 연결 자료구조 형태로 표현



❖ 이진 트리 순회(traversal)의 개념

- 모든 원소를 빠트리거나 중복하지 않고 처리하는 연산
 - 이진 트리 순회를 위한 세부 작업
 - 작업 D : 현재 노드를 방문하여 처리한다.
 - 작업 L : 현재 노드의 왼쪽 서브 트리로 이동한다.
 - 작업 R : 현재 노드의 오른쪽 서브 트리로 이동한다.



❖ 이진 트리 순회(traversal)의 개념

- 이진 트리가 순환적으로 정의되어 구성되어 있으므로, 순회작업도 서브 트리에 대해서 순환적으로 반복하여 완성한다.
- 왼쪽 서브 트리에 대한 순회를 오른쪽 서브 트리 보다 먼저 수행한다
- 순회의 종류
 - 전위 순회
 - 중위 순회
 - 후의 순회

❖ 전위순회(preorder traversal)

- $D \rightarrow L \rightarrow R$ 순서로, 현재 노드를 방문하여 처리하는 작업 D를 가장 먼저 수행
- 이진 트리의 전위 순회 작업순서

- ① 작업 D : 현재 노드 n 을 처리한다.
- ② 작업 L : 현재 노드 n 의 왼쪽 서브 트리로 이동한다.
- ③ 작업 R : 현재 노드 n 의 오른쪽 서브 트리로 이동한다.

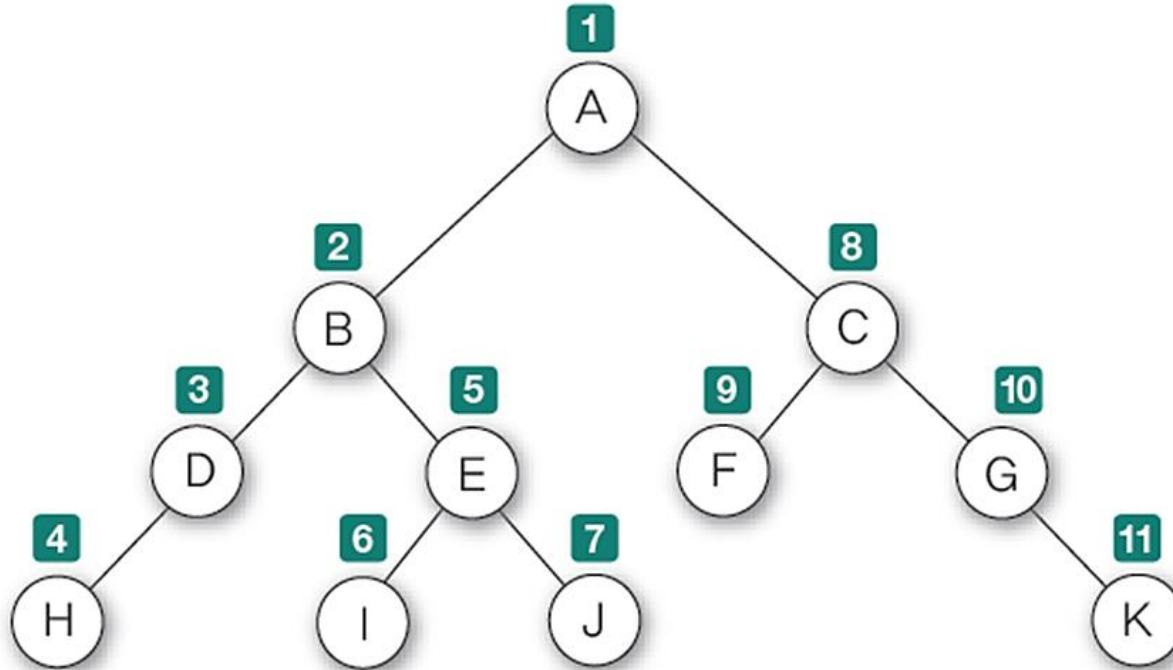
알고리즘

이진 트리의 전위 순회

```
preorder(T)
  if (T≠NULL) then {
    visit T.data;
    preorder(T.left);
    preorder(T.right);
  }
end preorder( )
```


❖ 전위순회(preorder traversal)

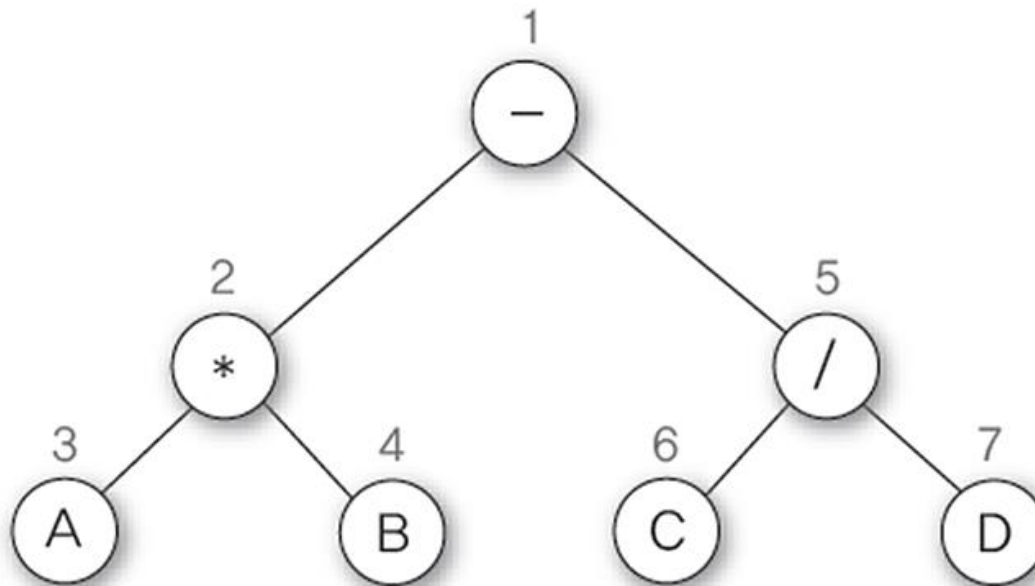
- 전위 순회의 예



전위 순회 경로 : A-B-D-H-E-I-J-C-F-G-K

❖ 순회의 종류 - 전위순회(preorder traversal)

- 수식 $A*B-C/D$ 를 이진 트리로 구성
 - 수식에 대한 이진 트리를 순회하면, 전위 표기 식을 구할 수 있음



수식에 대한 이진 트리의 전위 순회 경로 : $-*AB/CD$

❖ 중위 순회(inorder traversal)

- $L \rightarrow D \rightarrow R$ 순서로, 현재 노드를 방문하는 작업 D를 작업 L과 작업 R의 중간에 수행
- 이진 트리의 중위 순회 작업 순서

- ① 작업 L : 현재 노드 n의 왼쪽 서브 트리로 이동한다.
- ② 작업 D : 현재 노드 n을 처리한다.
- ③ 작업 R : 현재 노드 n의 오른쪽 서브 트리로 이동한다.

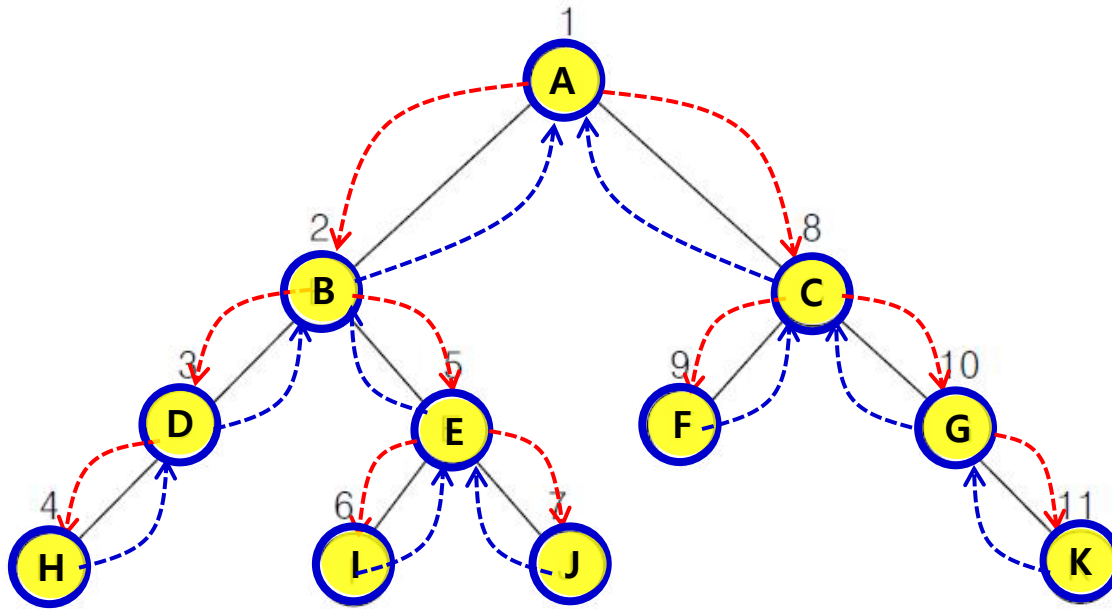
알고리즘

이진 트리의 중위 순회

```
inorder(T)
  if (T≠NULL) then {
    inorder(T.left);
    visit T.data;
    inorder(T.right);
  }
end inorder( )
```

❖ 중의 순회(inorder traversal)

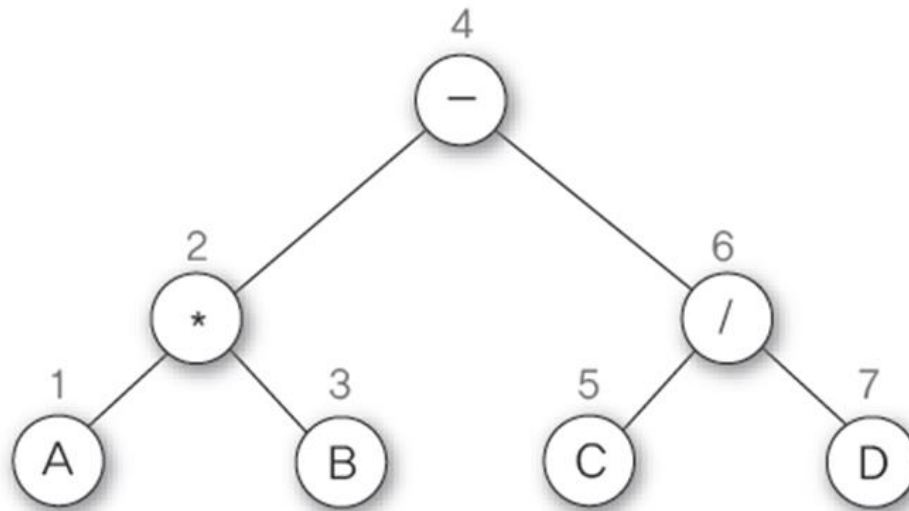
■ 중위 순회의 예



이진 트리의 중위 순회 경로 : H-D-B-I-E-J-A-F-C-G-K

❖ 중의 순회(inorder traversal)

- 수식 $A*B-C/D$ 를 이진 트리로 구성
 - 수식 이진 트리를 중위 순회하면, 수식에 대한 중위 표기식을 구할 수 있다.



수식에 대한 이진 트리의 중위 순회 경로 : $A*B-C/D$

❖ 후위 순회(postorder traversal)

- $L \rightarrow R \rightarrow D$ 순서로, 현재 노드를 방문하는 D 작업을 나중에 수행
- 이진 트리의 후위 순회 작업 순서

- ① 작업 L : 현재 노드 n 의 왼쪽 서브 트리로 이동한다.
- ② 작업 R : 현재 노드 n 의 오른쪽 서브 트리로 이동한다.
- ③ 작업 D : 현재 노드 n 을 처리한다.

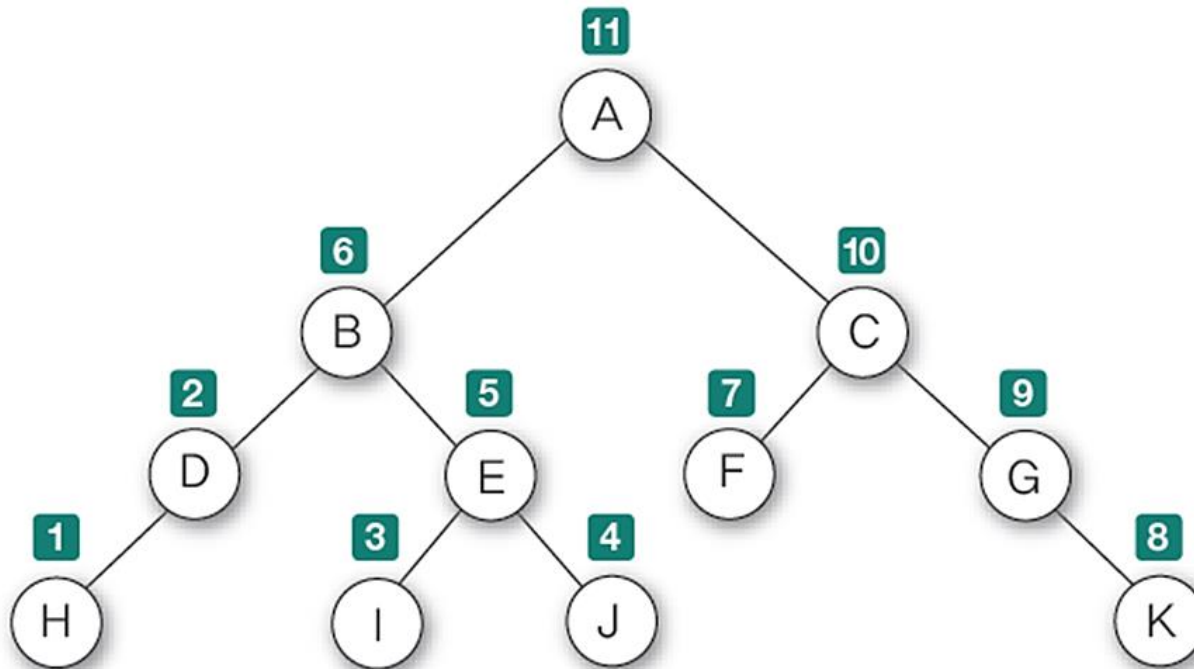
알고리즘

이진 트리의 후위 순회

```
postorder(T)
  if (T ≠ NULL) then {
    postorder(T.left);
    postorder(T.right);
    visit T.data;
  }
end postorder( )
```

❖ 후위 순회(postorder traversal)

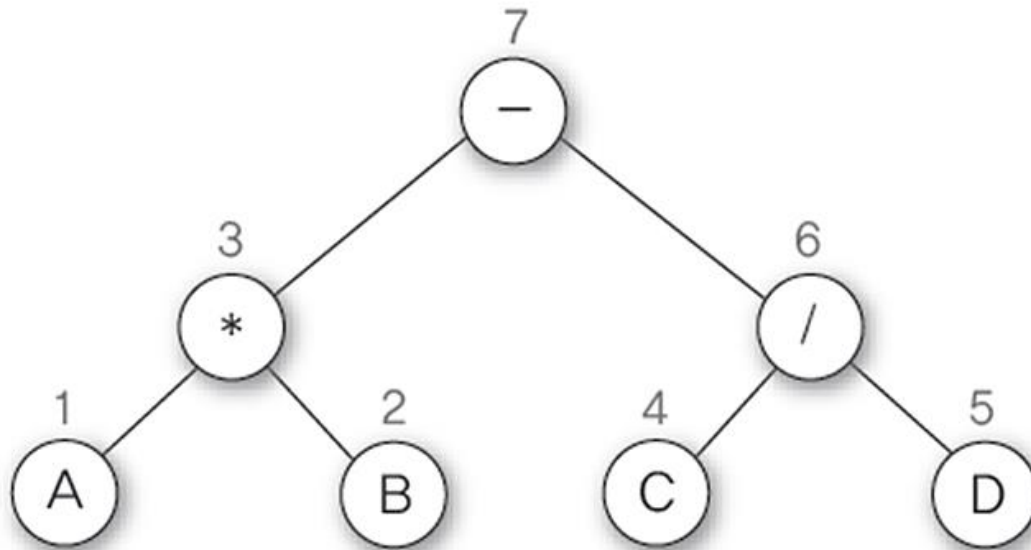
- 후위 순회의 예



이진 트리의 후위 순회 경로 : H-D-I-J-E-B-F-K-G-C-A

❖ 순회의 종류 - 후위 순회(postorder traversal)

- 수식 $A*B-C/D$ 를 이진 트리로 구성
 - 수식 이진 트리를 후위 순회하면, 수식에 대한 후위 표기 식을 구할 수 있다.



수식에 대한 이진 트리의 후위 순회 경로 : $AB*CD/-$

❖ 이진 트리 순회하기 프로그램

- 이진 트리를 연결 자료구조 방식으로 표현
- makeRootNode()함수를 이용해 트리를 구성하여 전위 순회, 중위 순회, 후위 순회를 수행한 결과를 출력하여 확인하는 프로그램
- 연결 자료구조의 노드를 정의

```
#include <stdio.h>
#include <stdlib.h>

// 연결 자료구조로 구성하기 위해 트리의 노드 정의
typedef struct treeNode {
    char data;
    struct treeNode *left;           // 왼쪽 서브 트리에 대한 링크 필드
    struct treeNode *right;         // 오른쪽 서브 트리에 대한 링크 필드
} treeNode;
```

❖ 이진 트리 순회하기 프로그램

- 왼쪽 서브 트리와 오른쪽 서브 트리를 연결하는 연산

```
// data를 루트 노드로 하여 왼쪽 서브 트리와 오른쪽 서브 트리를 연결하는 연산
treeNode* makeRootNode(char data, treeNode* leftNode, treeNode* rightNode)
{
    treeNode* root = (treeNode *)malloc(sizeof(treeNode));
    root->data = data;
    root->left = leftNode;
    root->right = rightNode;
    return root;
}

// 이진 트리에 대한 전위 순회 연산
void preorder(treeNode* root)
{
    if (root)
    {
        printf("%c ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}
```

❖ 이진 트리 순회하기 프로그램

■ 이진 트리의 중위, 후위 연산

```
// 이진 트리에 대한 중위 순회 연산
void inorder(treeNode* root)
{
    if (root) {
        inorder(root->left);
        printf("%c ", root->data);
        inorder(root->right);
    }
}

// 이진 트리에 대한 후위 순회 연산
void postorder(treeNode* root)
{
    if (root) {
        postorder(root->left);
        postorder(root->right);
        printf("%c ", root->data);
    }
}
```

❖ 이진 트리 순회하기 프로그램

■ 이진 트리의 단말 노드 검사 연산

```
// 이진 트리의 단말 노드 검사
bool is_leaf(treeNode* root)
{
    if (root->left == NULL && root->right == NULL)
        return true;
    else
        return false;
}

int get_Max(int a, int b)
{
    return a > b ? a : b;
}
```

❖ 이진 트리 순회하기 프로그램

■ 이진 트리의 레벨 검사 연산

```
// 이진 트리의 레벨 검사
int get_Depth(treeNode* root)
{
    if (is_leaf(root) || root == NULL)
        return 0;
    else
    {
        return get_Max(get_Depth(root->left), get_Depth(root->right)) + 1;
    }
}
```

❖ 이진 트리 순회하기 프로그램

- 이진 트리를 구성하고 순회 결과 확인

```
void main()
{
    // 이진 트리 구성하기
    treeNode* n11 = makeRootNode('K', NULL, NULL);
    treeNode* n10 = makeRootNode('G', NULL, n11);
    treeNode* n9 = makeRootNode('F', NULL, NULL);
    treeNode* n8 = makeRootNode('C', n9, n10);
    treeNode* n7 = makeRootNode('J', NULL, NULL);
    treeNode* n6 = makeRootNode('I', NULL, NULL);
    treeNode* n5 = makeRootNode('E', n6, n7);
    treeNode* n4 = makeRootNode('H', NULL, NULL);
    treeNode* n3 = makeRootNode('D', n4, NULL);
    treeNode* n2 = makeRootNode('B', n3, n5);
    treeNode* n1 = makeRootNode('A', n2, n8);
```

❖ 이진 트리 순회하기 프로그램

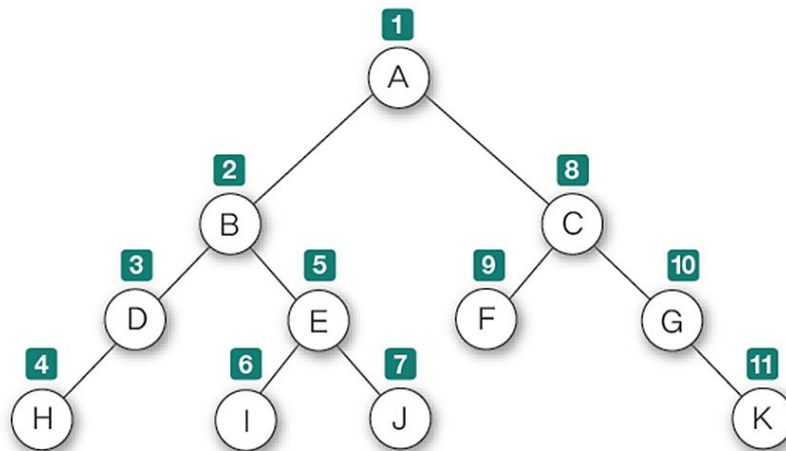
- 이진 트리를 구성하고 순회 결과 확인

```
printf("\n preorder : ");  
preorder(n1);  
getchar();  
  
printf("\n inorder : ");  
inorder(n1);  
getchar();  
  
printf("\n postorder : ");  
postorder(n1);  
getchar();  
  
printf("\n Depth : %d", get_Depth(n1));  
printf("\n Level : %d", get_Depth(n1) + 1);  
}
```

❖ 이진 트리 순회하기 프로그램

■ 실행 결과 확인

```
C:\한국산업기술대학\2021년1학기\데이터구조론\실습예...  
preorder : A B D H E I J C F G K
```



전위 순회 경로 : A-B-D-H-E-I-J-C-F-G-K

이진 트리의 순회

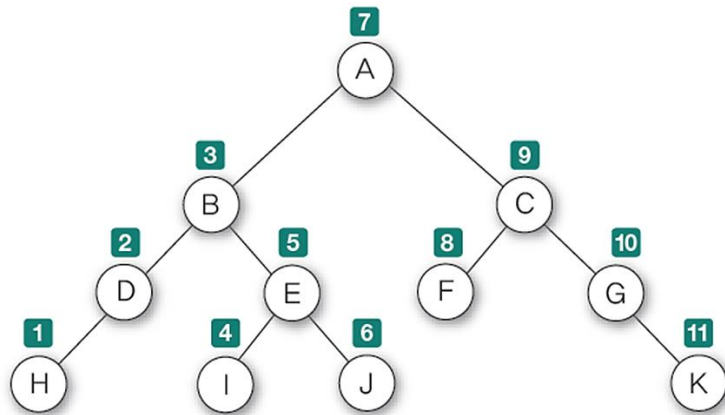
❖ 이진 트리 순회하기 프로그램

■ 실행 결과 확인

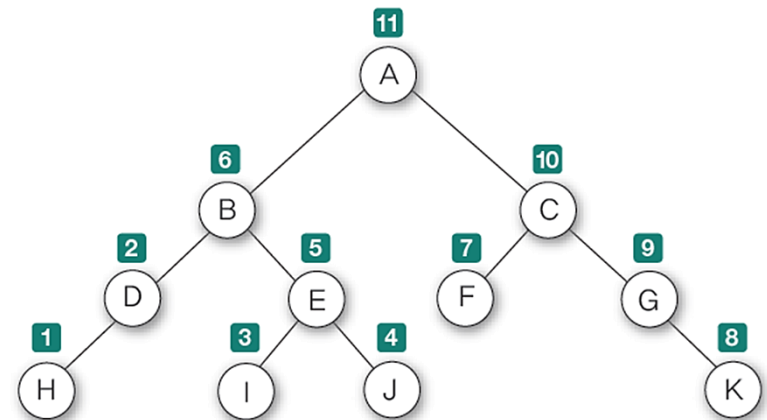
```
C:\한국산업기술대학교\2021년1학기\데이터구조\실습예제\이진트리순...
preorder : A B D H E I J C F G K
inorder  : H D B I E J A F C G K
postorder: H D I J E B F K G C A

Depth   : 3
Level   : 4

-----
Process exited after 257.3 seconds with return value 14
계속하려면 아무 키나 누르십시오 . . .
```



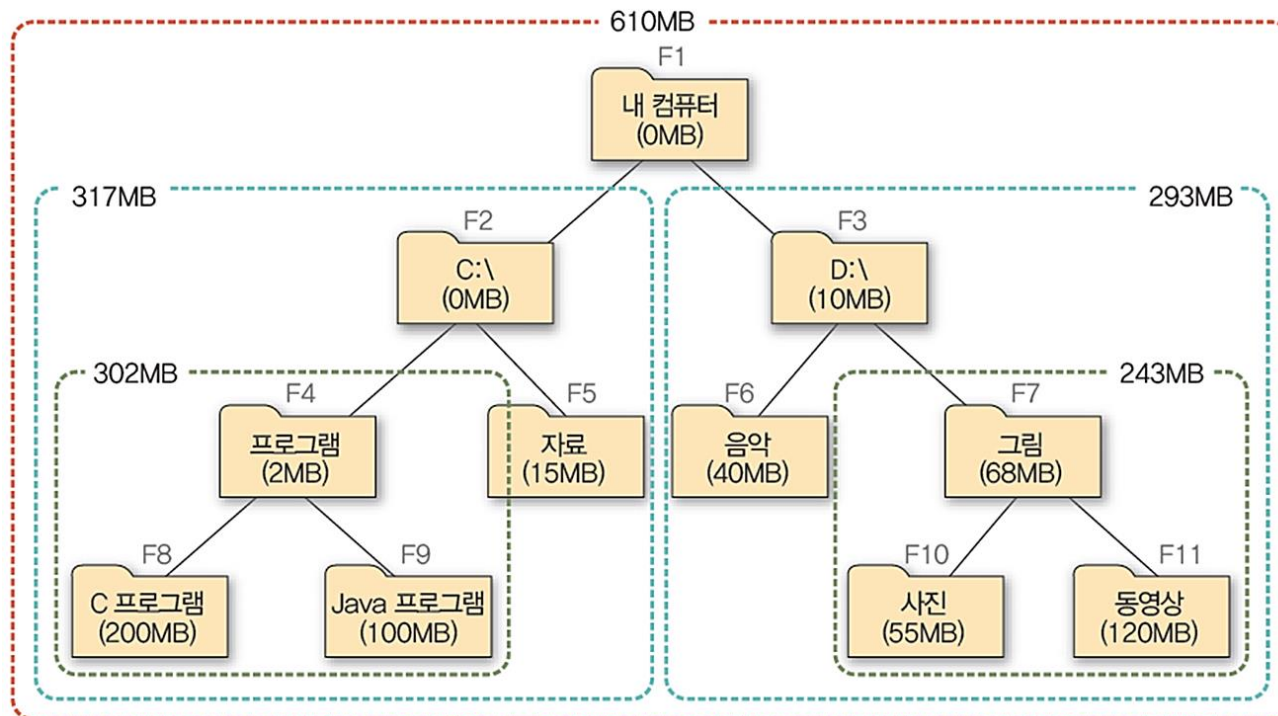
중위 순회 경로 : H-D-B-I-E-J-A-F-C-G-K



후위 순회 경로 : H-D-I-J-E-B-F-K-G-C-A

❖ 이진 트리 순회의 응용 프로그램

- 컴퓨터의 폴더 구조가 이진 트리 구조인 경우 각 폴더를 순회하면서 용량을 계산하면 내 컴퓨터의 전체 용량이 계산 가능함.
- 폴더 전체 용량은 폴더에 저장된 파일 용량과 하위 폴더의 용량을 합한 값이다. 상위 폴더 용량을 계산하려면 먼저 하위 폴더 용량을 계산해야 하므로 후위 순회를 사용한다.



❖ 이진 트리의 후위 순회를 이용 폴더 용량 계산 프로그램

```
#define _CRT_SECURE_NO_WARNINGS
#include "stdio.h"
#include "stdlib.h"

// 트리의 노드 구조 정의
typedef struct treeNode {
    int size;                // 데이터 필드
    struct treeNode *left;   // 왼쪽 서브 트리에 대한 링크 필드
    struct treeNode *right;  // 오른쪽 서브 트리에 대한 링크 필드
} treeNode;

int FolderSize = 0;
```

❖ 이진 트리의 후위 순회를 이용 폴더 용량 계산 프로그램

```
// 루트 노드에 왼쪽과 오른쪽 서브 트리를 연결하는 연산
treeNode* makeRootNode(int size, treeNode* leftNode, treeNode* rightNode)
{
    treeNode* root = (treeNode *)malloc(sizeof(treeNode));
    root->size = size;
    root->left = leftNode;
    root->right = rightNode;
    return root;
}

// 각 폴더 크기를 계산하기 위한 후위 순회 연산
int postorder_FolderSize(treeNode* root)
{
    if (root) {
        postorder_FolderSize(root->left);
        postorder_FolderSize(root->right);
        FolderSize += root->size;
    }
    return FolderSize;
}
```

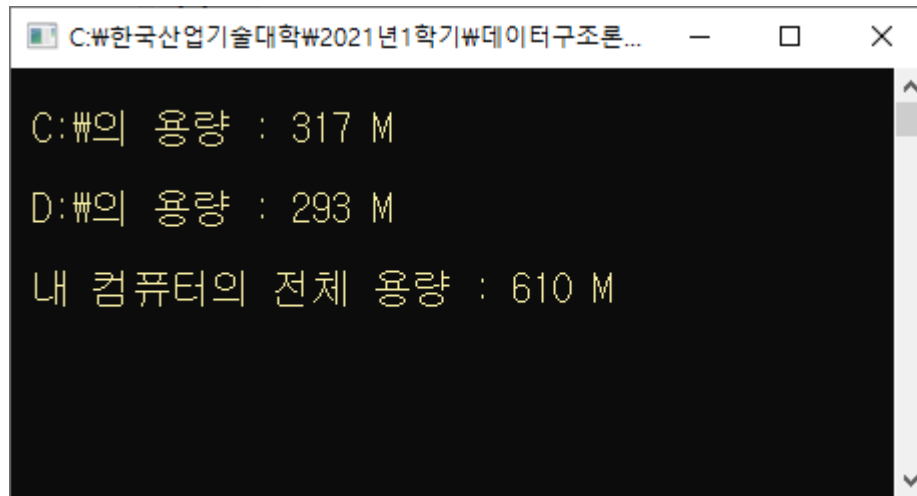
❖ 이진 트리의 후위 순회를 이용 폴더 용량 계산 프로그램

```
void main()
{
    treeNode* F11 = makeRootNode(120, NULL, NULL);
    treeNode* F10 = makeRootNode(55, NULL, NULL);
    treeNode* F9 = makeRootNode(100, NULL, NULL);
    treeNode* F8 = makeRootNode(200, NULL, NULL);
    treeNode* F7 = makeRootNode(68, F10, F11);
    treeNode* F6 = makeRootNode(40, NULL, NULL);
    treeNode* F5 = makeRootNode(15, NULL, NULL);
    treeNode* F4 = makeRootNode(2, F8, F9);
    treeNode* F3 = makeRootNode(10, F6, F7);
    treeNode* F2 = makeRootNode(0, F4, F5);
    treeNode* F1 = makeRootNode(0, F2, F3);

    printf("\n C:\\의 용량 : %d M \n", postorder_FolderSize(F2));
    FolderSize = 0;
    printf("\n D:\\의 용량 : %d M \n", postorder_FolderSize(F3));
    FolderSize = 0;
    printf("\n 내 컴퓨터의 전체 용량 : %d M \n", postorder_FolderSize(F1));
    getchar();
}
```

❖ 이진 트리의 후위 순회를 이용 폴더 용량 계산 프로그램 : [교재 340p](#)

■ 실행 결과 확인



```
C:\₩한국산업기술대학₩2021년1학기₩데이터구조론...  
C:₩의 용량 : 317 M  
D:₩의 용량 : 293 M  
내 컴퓨터의 전체 용량 : 610 M
```

❖ 스레드 이진 트리(Thread Binary Tree)

- 재귀호출 없이 순회할 수 있도록 수정한 이진 트리
 - 재귀호출 방식은 알고리즘이나 함수 구현은 간단하지만, 수행 성능 측면에서는 시스템 스택을 사용하면서 호출과 복귀를 관리해야 하고 이진 트리의 하위 레벨로 내려갈수록 재귀호출 깊이가 깊어지므로 비효율적임.
 - 스레드(Thread)
 - ⇒ 스레드 이진 트리에서 자식 노드가 없는 경우 링크 필드를 널 포인터 대신 순회 순서상의 다른 노드를 가리키도록 설정. 이런 링크 필드를 스레드라 함

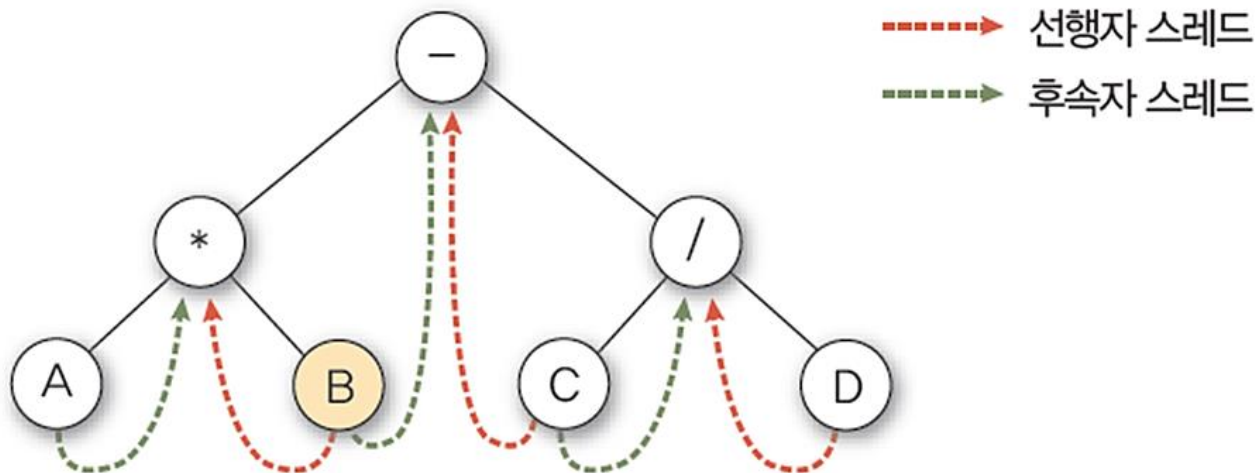
❖ 스레드 이진 트리(Thread Binary Tree)

- 이진 트리 노드 구조에 isThread 필드를 추가하여 정의
- isThread 필드는 링크 필드가 자식 노드에 대한 포인터인지 아니면 자식 노드 대신 스레드가 저장되어 있는지 구별하기 위한 태그 필드
- 스레드 이진 트리의 노드 정의

```
typedef struct treeNode {  
    char data;  
    struct treeNode *left;  
    struct treeNode *right;  
    int isThreadLeft;  
    int isThreadRight;  
} treeNode;
```

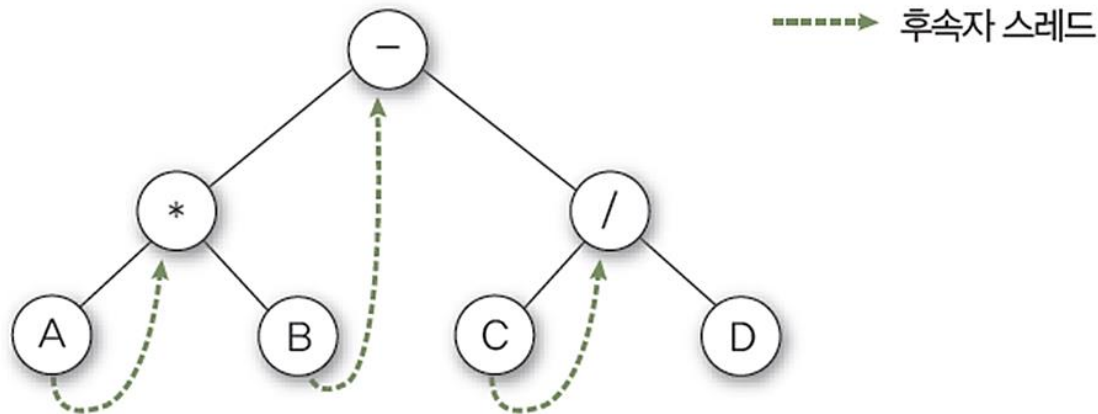

❖ 스레드 이진 트리(Thread Binary Tree)

- $A * B - C / D$ 수식의 이진 트리에서 중위 순회를 한다면, 왼쪽 스레드에는 선 행자를 설정하고 오른쪽 스레드에는 후 속자를 설정. 중위 순회 경로가 $A \rightarrow * \rightarrow B \rightarrow - \rightarrow C \rightarrow / \rightarrow D$ 이므로 B의 선 행자는 '*'이고 B의 후 속자는 '-'임
- 선행자/후속자 스레드를 갖는 스레드 이진트리



❖ 스레드 이진 트리(Thread Binary Tree)

- 순회를 위해서 후속자 정보만 필요한 경우에는 오른쪽 링크 필드만 스레드로 사용하는 스레드 이진 트리를 사용할 수 있다.
- 후속자 스레드를 갖는 스레드 이진 트리



❖ 스레드 이진 트리(Thread Binary Tree)

■ 트리 노드 구성

```
// data를 루트 노드로 하여 왼쪽 서브 트리와 오른쪽 서브 트리를 연결하는 연산
treeNode* makeRootNode(char data, treeNode* leftNode, treeNode* rightNode)
{
    treeNode* root = (treeNode *)malloc(sizeof(treeNode));
    root->data = data;
    root->left = leftNode;
    root->right = rightNode;
    return root;
}
```

```
// data를 루트 노드로 하여 왼쪽 서브 트리와 오른쪽 서브 트리를 연결하는 연산
treeNode* makeRootNode(char data, treeNode* leftNode, treeNode* rightNode, int isThreadRight)
{
    treeNode* root = (treeNode *)malloc(sizeof(treeNode));
    root->data = data;
    root->left = leftNode;
    root->right = rightNode;
    root->isThreadRight = isThreadRight;
    return root;
}
```

❖ 스레드 이진 트리(Thread Binary Tree)

■ 후속자 반환 연산과 스레드 중위 순회

```
// 후속자 노드를 반환하는 연산
treeNode *findThreadSuccessor(treeNode* p)
{
    treeNode *q = p->right;
    if (q == NULL)
        return q;
    if (p->isThreadRight == 1)
        return q;
    while (q->left != NULL)
        q = q->left;
    return q;
}
```

```
// 스레드 이진 트리의 중위 순회
void threadInorder(treeNode* root)
{
    treeNode * q;
    q = root;
    while (q->left)
        q = q->left;
    do {
        printf("%3c", q->data);
        q = findThreadSuccessor(q);
    } while (q);
}
```

❖ 스레드 이진 트리(Thread Binary Tree)

■ 메인함수 구성

```
void main()
{
    // (A*B-C/D) 수식 이진 트리 만들기
    treeNode* n7 = makeRootNode('D', NULL, NULL, 0);
    treeNode* n6 = makeRootNode('C', NULL, NULL, 1);
    treeNode* n5 = makeRootNode('B', NULL, NULL, 1);
    treeNode* n4 = makeRootNode('A', NULL, NULL, 1);
    treeNode* n3 = makeRootNode('/', n6, n7, 0);
    treeNode* n2 = makeRootNode('*', n4, n5, 0);
    treeNode* n1 = makeRootNode('-', n2, n3, 0);

    n4->right = n2;
    n5->right = n1;
    n6->right = n3;

    printf("\n 스레드 이진 트리의 중위 순회 : ");
    threadInorder(n1);

    getchar();
}
```

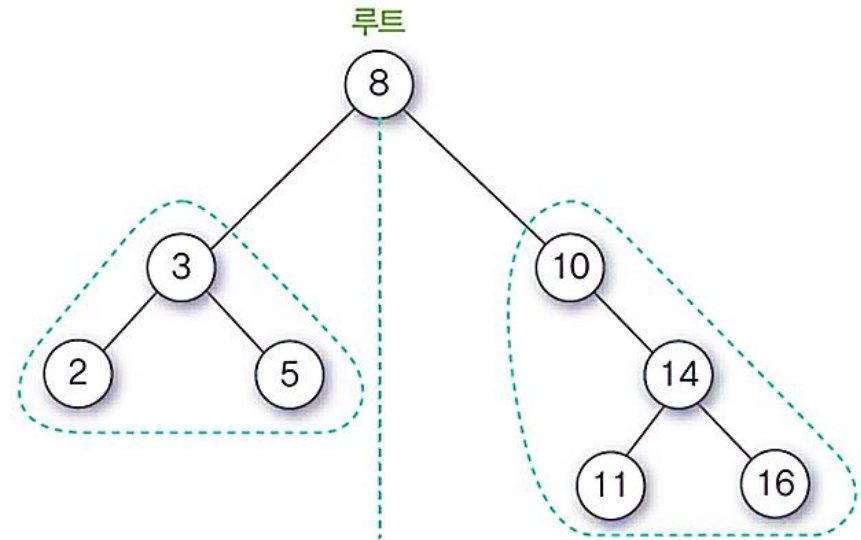
❖ 이진 탐색 트리(BST, Binary Search Tree)

- 이진 트리를 탐색용 자료구조로 사용하기 위해 원소 크기에 따라 노드 위치를 정의한 것이다.

이진 탐색 트리의 정의

- 모든 원소는 서로 다른 유일한 키를 갖는다.
- 왼쪽 서브 트리에 있는 원소들의 키는 그 루트의 키보다 작다.
- 오른쪽 서브 트리에 있는 원소들의 키는 그 루트의 키보다 크다.
- 왼쪽 서브 트리과 오른쪽 서브 트리도 이진 탐색 트리이다.

이진 탐색 트리 구조



왼쪽 서브 트리의 키 값 < 루트의 키 값 < 오른쪽 서브 트리의 키 값

❖ 이진 탐색 트리의 탐색 연산

- 루트에서 시작
- 탐색할 키 값 x 를 루트 노드의 키 값과 비교

노드 키값 비교	결과
키 값 $x =$ 루트 노드의 키 값	원하는 원소를 찾았으므로 탐색연산 성공
키 값 $x <$ 루트 노드의 키 값	루트 노드의 왼쪽 서브 트리에 대해서 탐색연산 수행
키 값 $x >$ 루트 노드의 키 값	루트 노드의 오른쪽 서브 트리에 대해서 탐색연산 수행

- 서브 트리에 대해서 순환적으로 탐색 연산을 반복

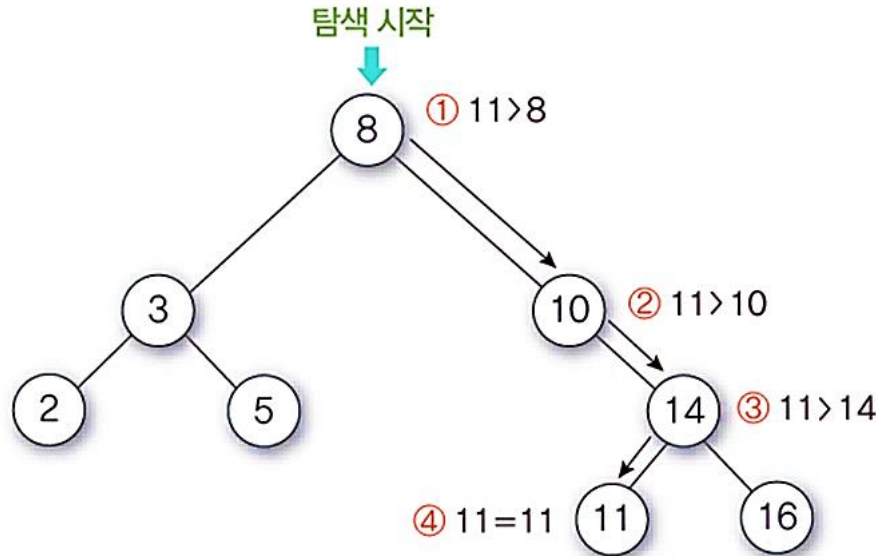
❖ 이진 탐색 트리의 탐색 연산

■ 탐색 연산 알고리즘

알고리즘	이진 탐색 트리의 노드 탐색
<pre>searchBST(bsT, x) p ← bsT; if (p=null) then return null; if (x = p.key) then return p; if (x < p.key) then return searchBST(p.left, x); else return searchBST(p.right, x); end searchBST()</pre>	

❖ 이진 트리의 탐색 연산

- 이진 탐색 트리에서 원소 11을 탐색



- ① (찾는 키값 11 > 루트 노드의 키값 8)이므로 오른쪽 서브 트리 탐색
- ② (찾는 키값 11 > 노드의 키값 10)이므로 다시 오른쪽 서브 트리 탐색
- ③ (찾는 키값 11 < 노드의 키값 14)이므로 왼쪽 서브 트리 탐색
- ④ (찾는 키값 11 = 노드의 키값 11)이므로 탐색 성공으로 연산 종료

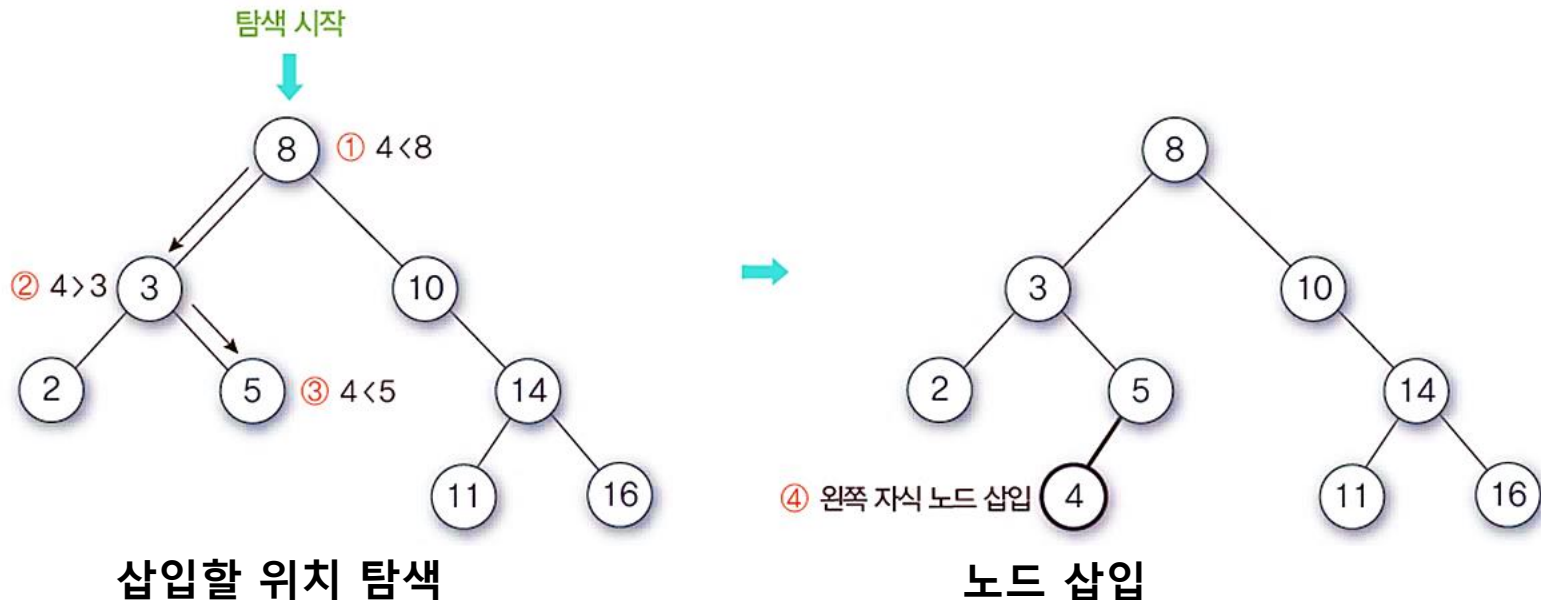
❖ 이진 탐색 트리의 삽입 연산

- 먼저 탐색 연산을 수행
 - 삽입할 원소와 같은 원소가 트리에 있으면 삽입할 수 없으므로, 같은 원소가 트리에 있는지 탐색하여 확인
 - 탐색에서 탐색 실패가 결정되는 위치가 삽입 위치가 됨
- 탐색 실패한 위치에 원소를 삽입

❖ 이진 탐색 트리의 삽입 연산 알고리즘

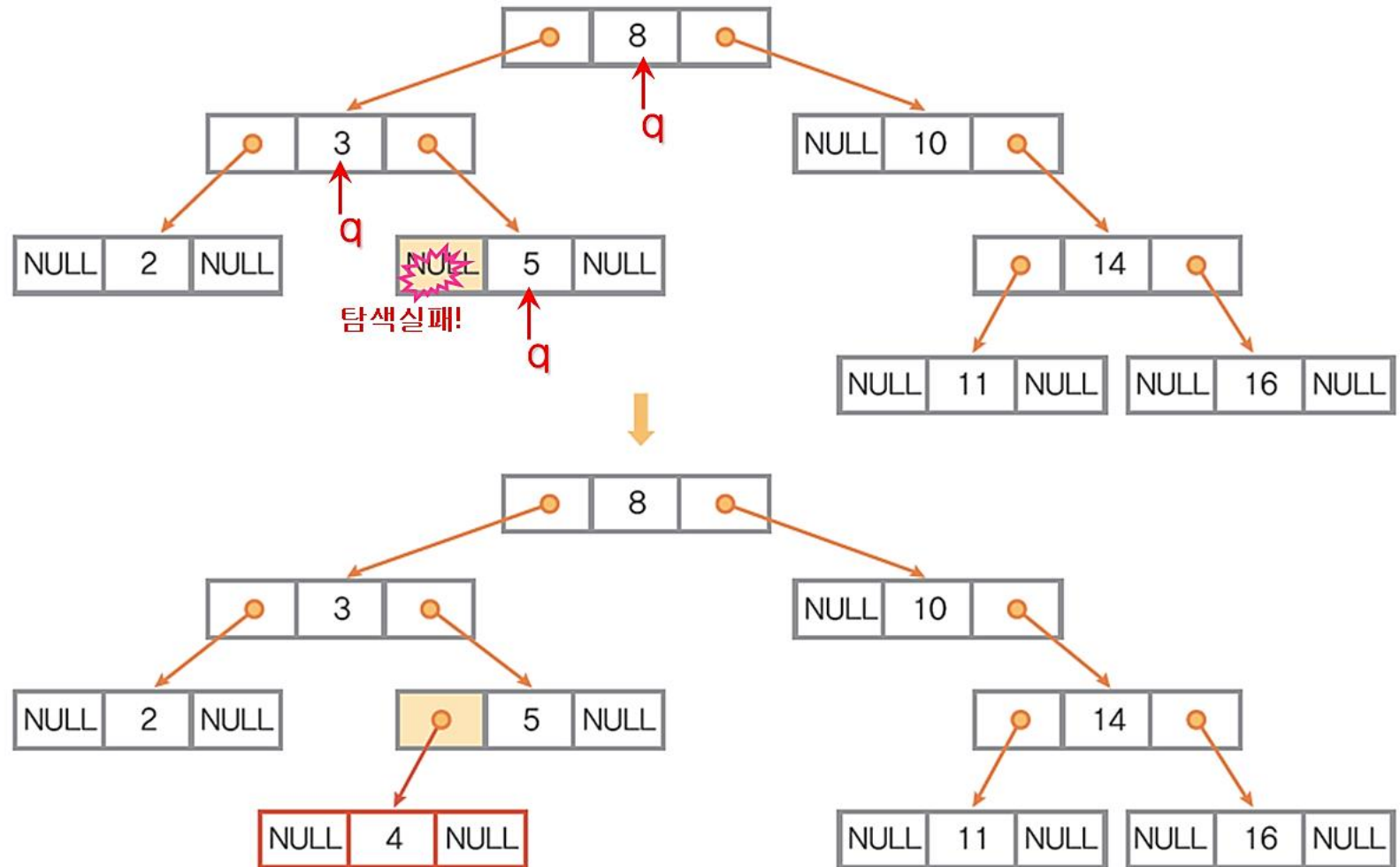
알고리즘	이진 탐색 트리의 노드 삽입
<pre>insertBST(bsT, x) p ← bsT; while (p ≠ null) do { if (x = p.key) then return; q ← p; if (x < p.key) then p ← p.left; else p ← p.right; }</pre>	} 삽입할 노드 탐색
<pre>new ← getNode(); new.key ← x; new.left ← null; new.right ← null;</pre>	} 삽입할 노드 생성
<pre>if (bsT = null) then bsT ← new; else if (x < q.key) then q.left ← new; else q.right ← new; return; end insertBST()</pre>	} 삽입 노드 연결

❖ 이진 탐색 트리에 원소 4를 삽입 하기



- ① (찾는 키값 $4 <$ 루트 노드의 키값 8)이므로 왼쪽 서브 트리를 탐색
- ② (찾는 키값 $4 >$ 노드의 키값 3)이므로 오른쪽 서브 트리를 탐색
- ③ (찾는 키값 $4 <$ 노드의 키값 5)이므로 왼쪽 서브 트리를 탐색해야 하지만, 왼쪽 자식 노드가 없으므로 노드 5의 왼쪽 자식 노드에서 탐색 실패가 발생
- ④ 실패가 발생한 자리, 즉 노드 5의 왼쪽 자식 노드 자리에 노드 4를 삽입

❖ 이진 탐색 트리에 원소 4를 삽입하는 과정을 연결 자료구조로 표현

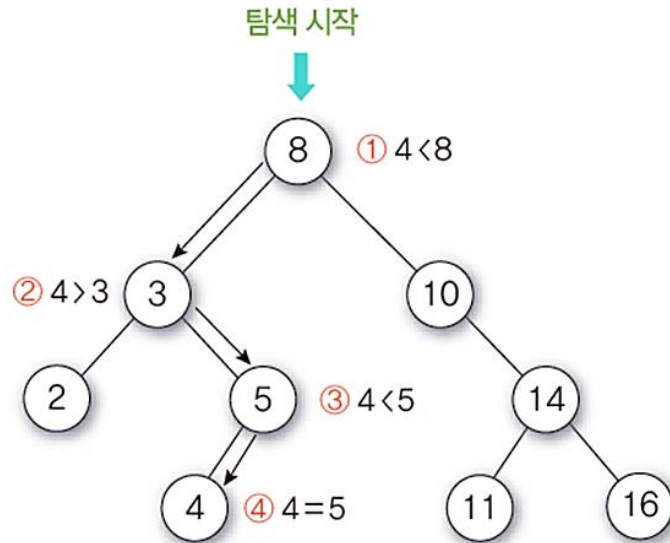


❖ 이진 탐색 트리의 삭제 연산

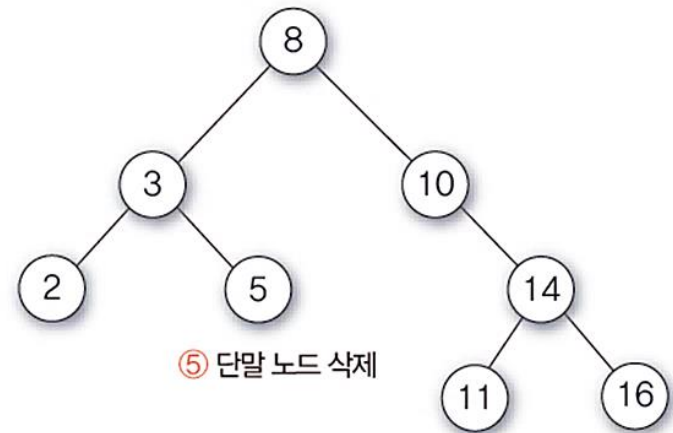
- 먼저 탐색 연산을 수행
 - 삭제할 노드의 위치를 알아야 하므로 트리를 탐색
- 탐색하여 찾은 노드를 삭제
 - 노드의 삭제 후에도 이진 탐색 트리를 유지해야 하므로 삭제 노드의 경우에 대한 후속 처리(이진 탐색 트리의 재구성 작업)가 필요함
 - ☞ 삭제할 노드의 경우
 - 삭제할 노드가 단말 노드인 경우(차수 = 0)
 - 삭제할 노드가 자식 노드를 한 개 가진 경우(차수 = 1)
 - 삭제할 노드가 자식 노드를 두 개 가진 경우(차수 = 2)

❖ 삭제할 노드가 단말 노드의 경우(차수 = 0)의 삭제 연산

- 노드 4를 삭제하는 경우

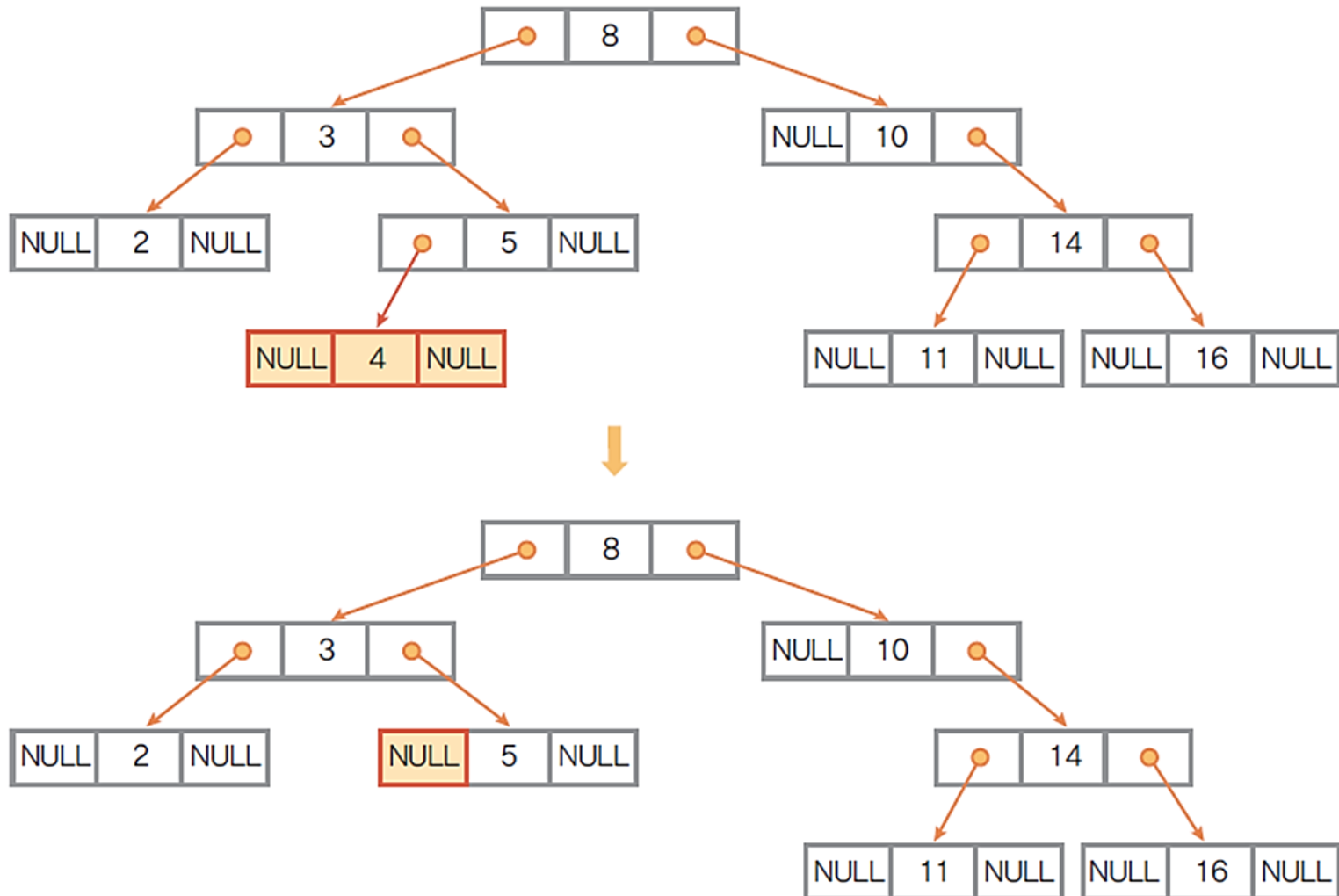


삭제할 노드 탐색

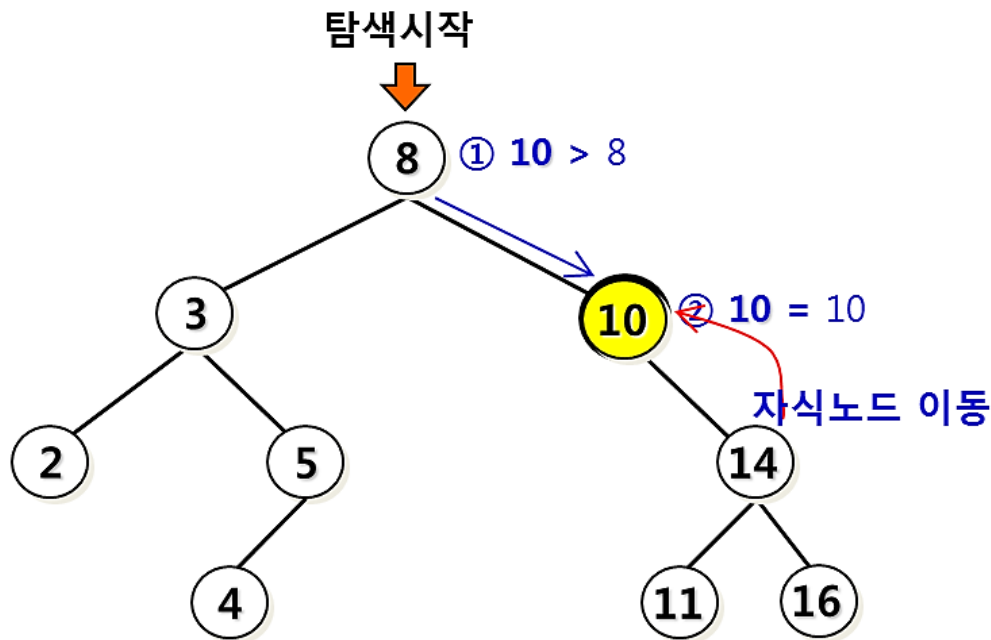


노드 4 삭제

❖ 4를 삭제하기 전과 후로 연결 자료구조를 표현

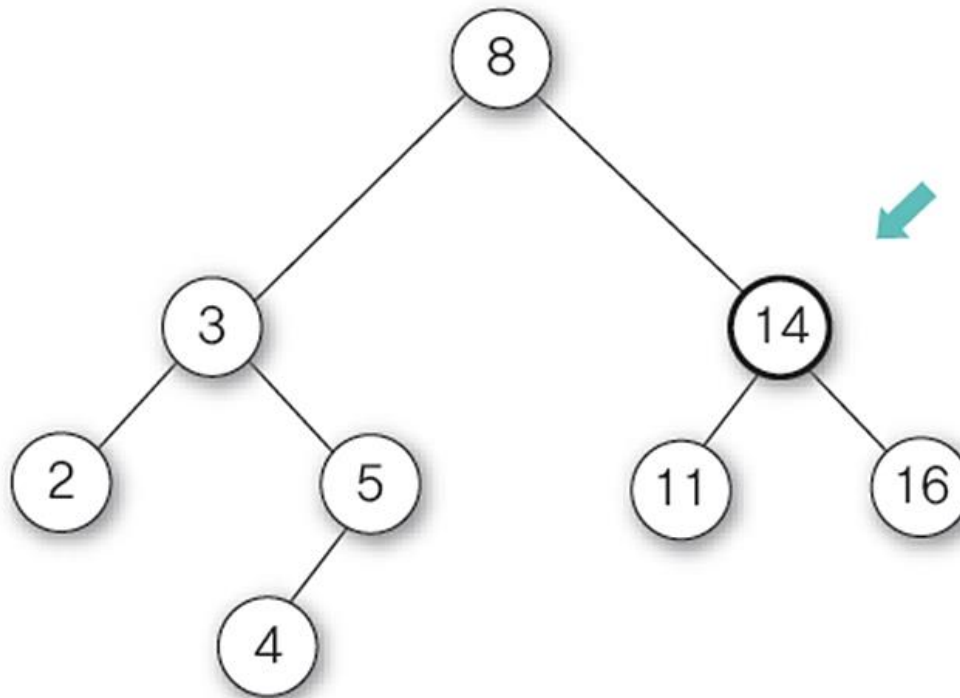


- ❖ 삭제할 노드가 자식 노드를 한 개 가진 경우(차수 = 1)의 삭제 연산
 - 노드를 삭제하면, 자식 노드는 트리에서 연결이 끊어져서 고아가 됨
 - 후속 처리 : 삭제한 부모노드의 자리를 자식노드에게 물려줌
 - 노드 10을 삭제하는 경우



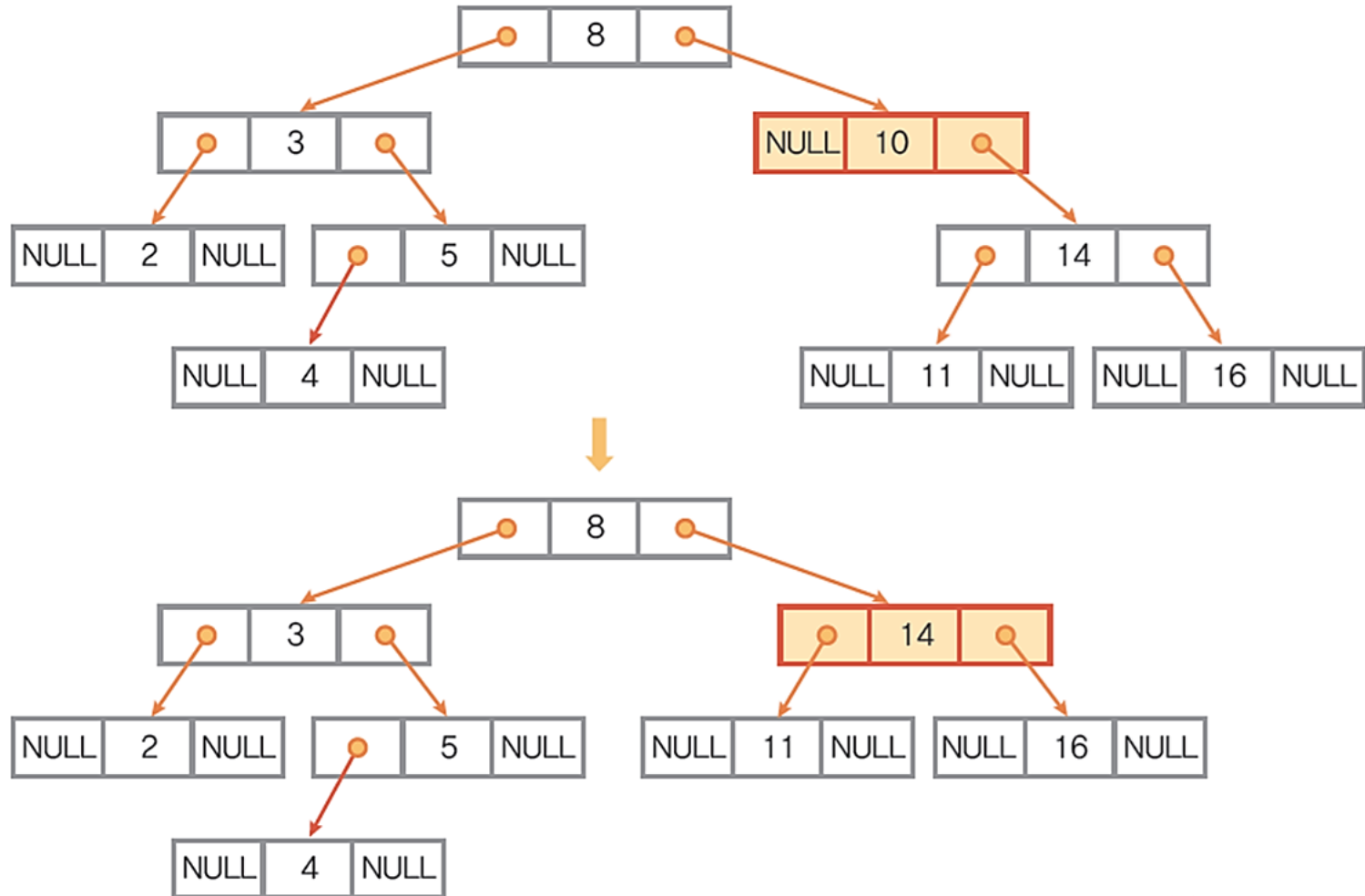
- 1단계: 삭제할 노드 **탐색**
- 2단계: 탐색한 노드 **삭제**
- 3단계: **후속처리**

- ❖ 삭제할 노드가 자식 노드를 한 개 가진 경우(차수 = 1)의 삭제 연산
 - 노드 10을 삭제하는 경우



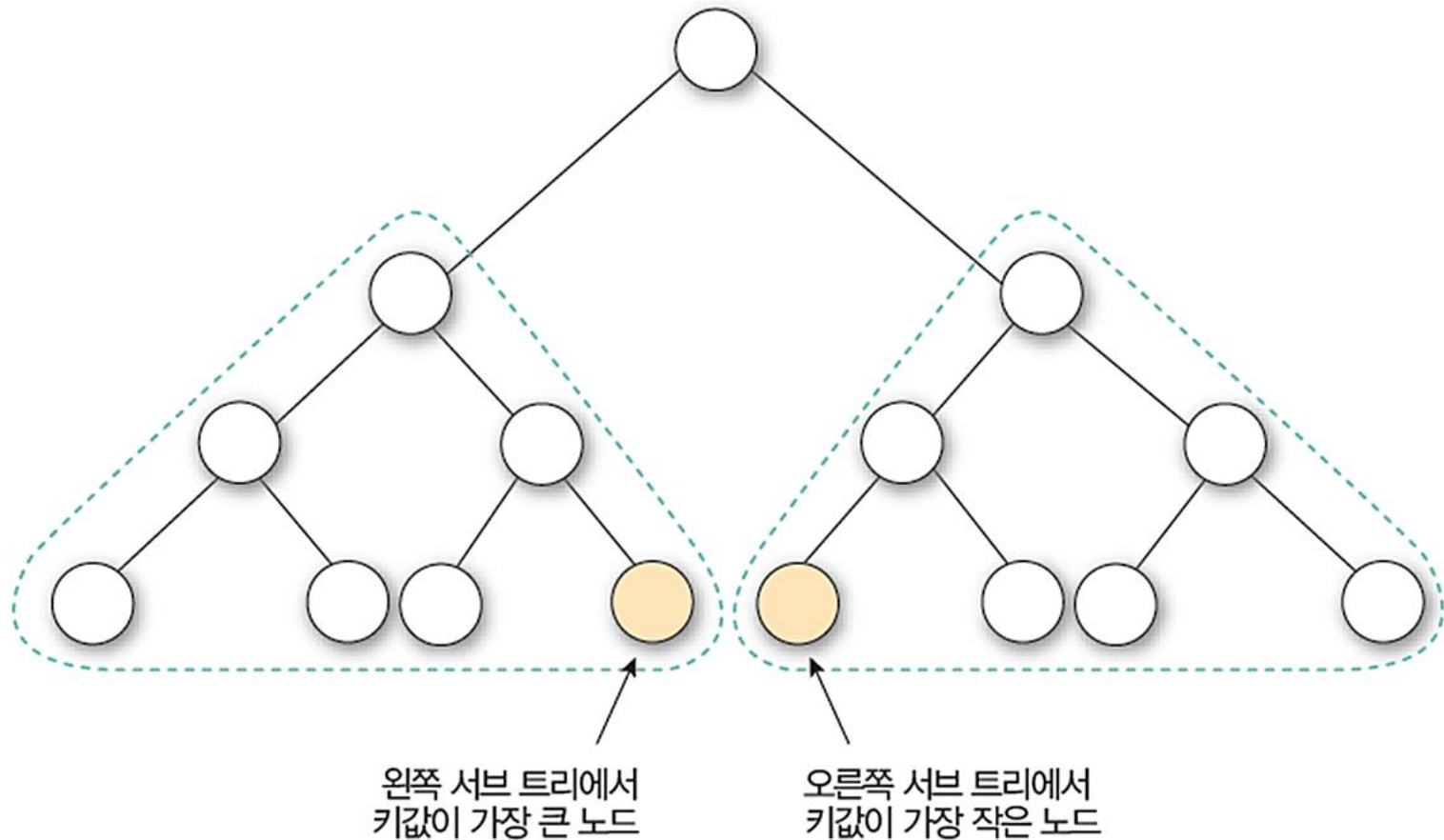
자식 노드의 위치 조정

- ❖ 삭제할 노드가 자식 노드를 한 개 가진 경우(차수 = 1)의 삭제 연산
 - 노드 10을 삭제하는 경우에 대한 단순 연결 리스트 표현



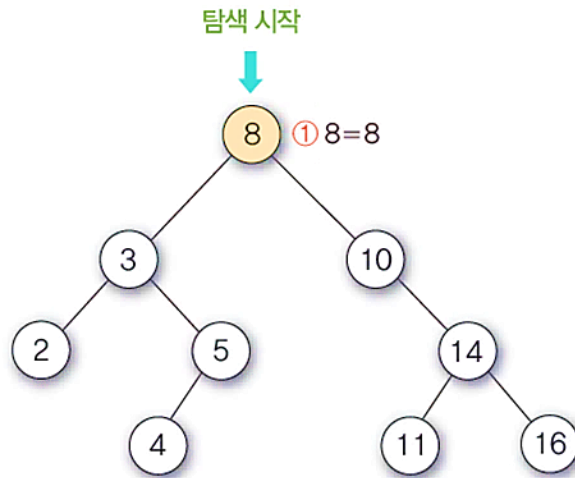
- ❖ 삭제할 노드가 자식 노드를 두 개 가진 경우(차수 = 2)의 삭제 연산
 - 노드를 삭제하면, 자식 노드들은 트리에서 연결이 끊어져서 고아가 됨
 - 후속 처리 : 삭제한 노드의 자리를 자손 노드들 중에서 선택한 후계자에게 물려줌
 - 후계자 선택 방법
 - 왼쪽 서브트리에서 가장 큰 자손노드 선택
왼쪽 서브트리의 오른쪽 링크를 따라 계속 이동하여 오른쪽 링크 필드가 NULL인 노드 즉, 가장 오른쪽 노드가 후계자가 됨
 - 오른쪽 서브트리에서 가장 작은 자손노드 선택
오른쪽 서브트리에서 왼쪽 링크를 따라 계속 이동하여 왼쪽 링크 필드가 NULL인 노드 즉, 가장 왼쪽에 있는 노드가 후계자가 됨

- ❖ 삭제할 노드가 자식 노드를 두 개 가진 경우(차수 = 2)의 삭제 연산
 - 삭제한 노드의 자리를 물려 받을 수 있는 자손 노드

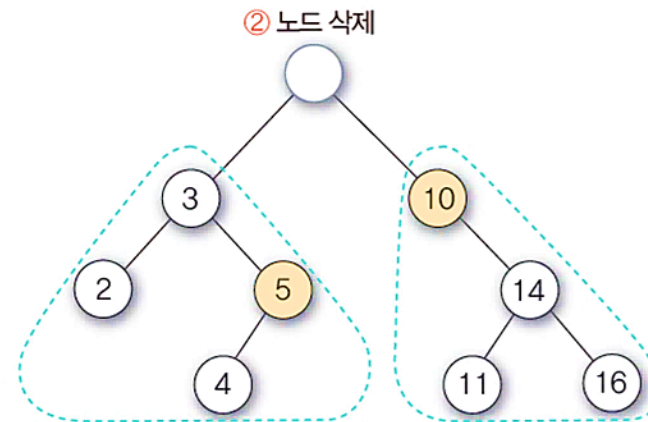


❖ 삭제할 노드가 자식 노드를 두 개 가진 경우(차수 = 2)의 삭제 연산

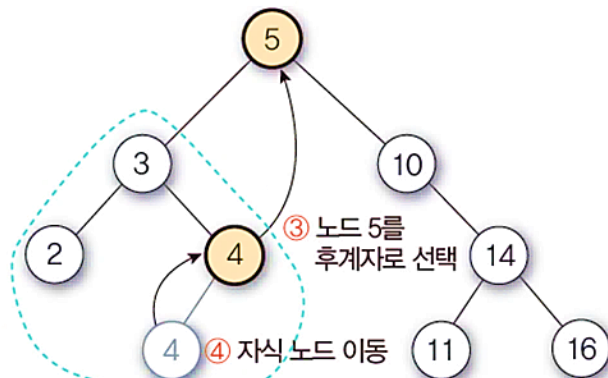
■ 노드 8을 삭제하는 경우



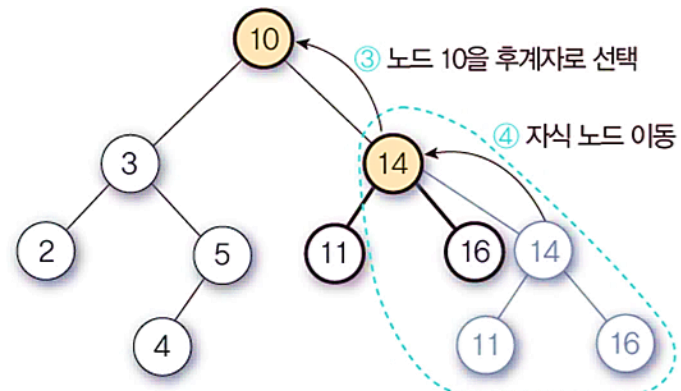
(a) 삭제할 노드 탐색



(b) 노드 삭제



(c)-1 노드 5를 후계자로 선택하는 경우

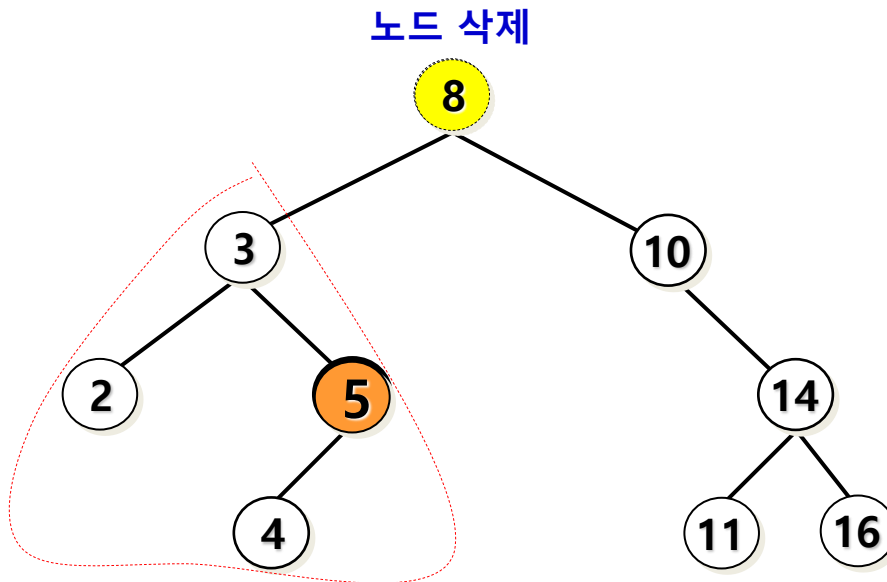


(c)-2 노드 10을 후계자로 선택하는 경우

❖ 삭제할 노드가 자식 노드를 두 개 가진 경우(차수 = 2)의 삭제 연산

■ 노드 5를 후계자로 선택한 경우

- ① 후계자 노드 5를 원래 자리에서 삭제하여, 삭제노드 8의 자리를 물려줌
- ② 후계자 노드 5의 원래 자리는 자식노드 4에게 물려주어 이진 트리를 재구성(자식 노드가 하나인 노드 삭제연산의 후속처리 수행)



1단계: 노드 삭제

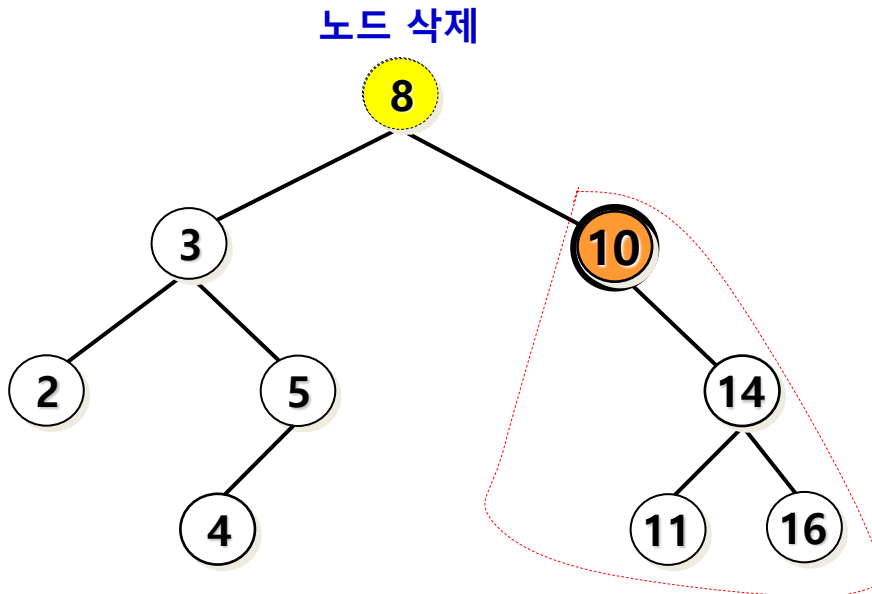
2단계: 삭제한 노드의 자리를
후계자에게 물려주기

3단계: 후계자노드의 원래자리를
자식노드에게 물려주기

❖ 삭제할 노드가 자식 노드를 두 개 가진 경우(차수 = 2)의 삭제 연산

■ 노드 10을 후계자로 선택한 경우

- ① 후계자 노드 10을 원래 자리에서 삭제하여, 삭제노드 8의 자리를 물려줌
- ② 후계자 노드 10의 원래 자리는 자식노드 14에게 물려주어 이진 트리를 재구성(자식 노드가 하나인 노드 삭제연산의 후속 처리 수행)



1단계: 노드 삭제

2단계: 삭제한 노드의 자리를
후계자에게 물려주기

3단계: 후계자노드의 원래자리를
자식노드에게 물려주기

❖ 이진 탐색 트리에서의 삭제 연산 알고리즘

알고리즘	이진 탐색 트리의 노드 삭제
<pre>deleteBST(bsT, x) p ← 삭제할 노드; parent ← 삭제할 노드의 부모 노드; // 삭제할 노드가 없는 경우 if (p = null) then return; // 삭제할 노드의 차수가 0인 경우 if (p.left = null and p.right = null) then { if (parent.left = p) then parent.left ← null; else parent.right ← null; }</pre>	

❖ 이진 탐색 트리에서의 삭제 연산 알고리즘

알고리즘	이진 탐색 트리의 노드 삭제
<pre>// 삭제할 노드의 차수가 1인 경우 else if (p.left = null or p.right = null) then { if (p.left ≠ null) then { if (parent.left = p) then parent.left ← p.left; else parent.right ← p.left; } else { if (parent.left = p) then parent.left ← p.right; else parent.right ← p.right; } } // 삭제할 노드의 차수가 2인 경우 else { q ← maxNode(p.left); // 왼쪽 서브 트리에서 후계자 노드를 포인터 q로 지정 p.key ← q.key; deleteBST(p.left, p.key); // 후계자 노드 자리에 대한 2차 재구성 } end deleteBST()</pre>	

❖ 이진 탐색 트리의 응용 프로그램 : [교재 570p](#)

```
#define _CRT_SECURE_NO_WARNINGS
#include "stdio.h"
#include "stdlib.h"
#include "string.h"

#define MAX_WORD_LENGTH 20
#define MAX_MEAN_LENGTH 200

// 영어 사전 항목의 구조 정의
typedef struct {
    char word[MAX_WORD_LENGTH];
    char mean[MAX_MEAN_LENGTH];
}element;

// 영어 사전 이진 트리의 노드 구조 정의
typedef struct treeNode {
    element key;
    struct treeNode *left;
    struct treeNode *right;
}treeNode;
```

❖ 이진 탐색 트리의 응용 프로그램

```
// 포인터 p가 가리키는 노드와 비교하여 항목 key를 삽입하는 연산
treeNode* insertKey(treeNode *p, element key)
{
    treeNode *newNode;
    int compare;

    // 삽입할 자리에 새 노드를 구성하여 연결
    if (p == NULL) {
        newNode = (treeNode*)malloc(sizeof(treeNode));
        newNode->key = key;
        newNode->left = NULL;
        newNode->right = NULL;
        return newNode;
    }
```

❖ 이진 탐색 트리의 응용 프로그램

```
// 이진 트리에서 삽입할 자리 탐색
else {
    compare = strcmp(key.word, p->key.word);
    if (compare < 0)
        p->left = insertKey(p->left, key);
    else if (compare > 0)
        p->right = insertKey(p->right, key);
    else
        printf("\n 이미 같은 단어가 있습니다! \n");

    return p; // 삽입한 자리 반환
}
}
```

❖ 이진 탐색 트리의 응용 프로그램

```
void insert(treeNode** root, element key)
{
    *root = insertKey(*root, key);
}

// root 노드부터 탐색하여 key와 같은 노드를 찾아 삭제하는 연산
void deleteNode(treeNode *root, element key)
{
    treeNode *parent, *p, *succ, *succ_parent;
    treeNode *child;

    parent = NULL;
    p = root;
    while ((p != NULL) && (strcmp(p->key.word, key.word) != 0))
    {
        parent = p;
        if (strcmp(key.word, p->key.word) < 0)
            p = p->left;
        else
            p = p->right;
    }
}
```

❖ 이진 탐색 트리의 응용 프로그램

```
// 삭제할 노드가 없는 경우
if (p == NULL) {
    printf("\n 삭제할 단어가 사전에 없습니다!!");
    return;
}

// 삭제할 노드가 단말 노드인 경우
if ((p->left == NULL) && (p->right == NULL)) {
    if (parent != NULL) {
        if (parent->left == p)
            parent->left = NULL;
        else
            parent->right = NULL;
    }
    else
        root = NULL;
}
```

❖ 이진 탐색 트리의 응용 프로그램

```
// 삭제할 노드가 자식 노드를 한 개 가진 경우
else if ((p->left == NULL) || (p->right == NULL)) {
    if (p->left != NULL)
        child = p->left;
    else
        child = p->right;

    if (parent != NULL) {
        if (parent->left == p)
            parent->left = child;
        else
            parent->right = child;
    }
    else
        root = child;
}
```


❖ 이진 탐색 트리의 응용 프로그램

```
// 삭제할 노드가 자식 노드를 두 개 가진 경우
else {
    succ_parent = p;
    succ = p->right;
    while (succ->left != NULL)
    {
        succ_parent = succ;
        succ = succ->left;
    }
    if (succ_parent->left == succ)
        succ_parent->left = succ->right;
    else
        succ_parent->right = succ->right;

    p->key = succ->key;
    p = succ;
}
free(p);
}
```

❖ 이진 탐색 트리의 응용 프로그램

```
// 이진 탐색 트리에서 키값이 key인 노드 위치를 탐색하는 연산
treeNode* searchBST(treeNode* root, element key)
{
    treeNode* p;
    int compare;
    p = root;

    while (p != NULL)
    {
        compare = strcmp(key.word, p->key.word);
        if (compare < 0)
            p = p->left;
        else if (compare > 0)
            p = p->right;
        else {
            printf("\n찾은 단어 : %s", p->key.word);
            return p;
        }
    }
    return p;
}
```

❖ 이진 탐색 트리의 응용 프로그램

```
// 이진 탐색 트리를 중위 순회하면서 출력하는 연산
void displayInorder(treeNode* root)
{
    if (root) {
        displayInorder(root->left);
        printf("\n%s : %s", root->key.word, root->key.mean);
        displayInorder(root->right);
    }
}

// 영어 사전 메뉴
void menu()
{
    printf("\n*-----*");
    printf("\n\t1 : 출력");
    printf("\n\t2 : 입력");
    printf("\n\t3 : 삭제");
    printf("\n\t4 : 검색");
    printf("\n\t5 : 종료");
    printf("\n*-----*\n  ");
}
```

❖ 이진 탐색 트리의 응용 프로그램

```
void main()
{
    element e;
    treeNode *root = NULL, *temp = NULL;
    int choice;

    // [5 :종료] 메뉴를 선택할 때까지 메뉴에 대한 영어 사전 동작 반복
    do {
        menu();
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                printf("\t[사전 출력]");
                displayInorder(root);
                printf("\n\t[사전 끝]\n");
                break;
```

❖ 이진 탐색 트리의 응용 프로그램

```
case 2:
    printf("\n[단어 입력] 단어를 입력하시오 : ");
    scanf("%s", e.word);
    printf("\n[단어입력] 단어의 뜻을 입력하시오 : ");
    scanf("%s", e.mean);
    insert(&root, e);
    break;
case 3:
    printf("\n[단어 삭제] 삭제할 단어 : ");
    scanf("%s", e.word);
    deleteNode(root, e);
    break;
```

❖ 이진 탐색 트리의 응용 프로그램

```
case 4:
    printf("\n[단어 검색] 검색할 단어 : ");
    scanf("%s", e.word);
    temp = searchBST(root, e);
    if (temp != NULL)
        printf("\n단어 뜻 : %s", temp->key.mean);
    else
        printf("\n사전에 없는 단어입니다.");
    break;
}
while (getchar() != '\n');
} while (choice != 5);
}
```